



Relatório Técnico

Asclépio — Assistente Clínico Inteligente

LLM fine-tunada · LangChain/LangGraph · Guardrails · RAG com fontes · Auditoria

TECH CHALLENGE · FASE 3 · FIAP PÓS-TECH IA PARA DEVS (8IADT)

Repositório: <https://github.com/rodrigogrosa/asclepio>

Gerado em 02/09/2026

Relatório Técnico — Asclépio

Tech Challenge · Fase 3 · FIAP Pós-Tech IA para Devs (8IADT)

Hospital Universitário FIAP (fictício) · Assistente clínico com LLM fine-tunada, LangChain/LangGraph, guardrails, RAG com fontes, anonimização e auditoria. Repositório: ver `README.md` · Vídeo: roteiro em `docs/ROTEIRO_VIDEO.md`.

Sumário

- 1. Contexto e objetivos
- 2. Visão da solução
- 3. Dados: base de conhecimento e dados sintéticos
- 4. Pré-processamento, anonimização e curadoria
- 5. Fine-tuning da LLM
- 6. Assistente médico com LangChain
- 7. Fluxos automatizados com LangGraph
- 8. Segurança, validação e explicabilidade
- 9. Avaliação do modelo e análise dos resultados
- 10. Engenharia: organização, DevEx, testes e operação
- 11. Limitações e próximos passos
- 12. Conclusão

Checklist de conformidade com o edital da Fase 3

Exigência do PDF	Onde está neste relatório	Evidência no sistema/repositório
Fine-tuning de LLM com protocolos, FAQs e modelos de laudos/receitas/procedimentos	§3, §5	<code>ml/</code> (pipeline), <code>ml/registry.json</code> , modelo <code>asclepio-med</code> no Ollama, tela IA & Modelos
Preparo dos dados com pré-processamento, anonimização e curadoria	§4	<code>data/processed/DATASET_CARD.md</code> , <code>asclepio_core/anonymizer.py</code> , testes
Assistente com LangChain: pipeline com a LLM customizada	§6	<code>services/assistant.py</code> (grafo), <code>/assistente</code>
Consultas em base estruturada (prontuários/registros)	§6	<code>services/patients.py</code> , <code>/pacientes</code> , Postgres
Respostas contextualizadas com dados atualizados do paciente	§6	"ver contexto anonimizado" no chat
Fluxos automatizados e seguros (exames pendentes, sugestões, alertas) coordenados com LangChain/LangGraph	§7	<code>services/workflow.py</code> , <code>/fluxos</code> , <code>/alertas</code>
Limites de atuação (nunca prescrever sem validação humana)	§8	<code>asclepio_core/guardrails.py</code> , nó <code>human_review</code> , testes
Logging detalhado para rastreamento e auditoria	§8, §10	trilha <code>audit_log</code> (hash chain), structlog, Prometheus, Langfuse, <code>/auditoria</code>
Explainability (fonte da informação na resposta)	§6, §8, §9	citações <code>[n]</code> , painel de fontes, métrica <code>citation_rate</code>
Código modularizado em Python + README completo	§10	<code>packages/</code> , <code>backend/</code> , <code>ml/</code> , <code>README.md</code> , CI

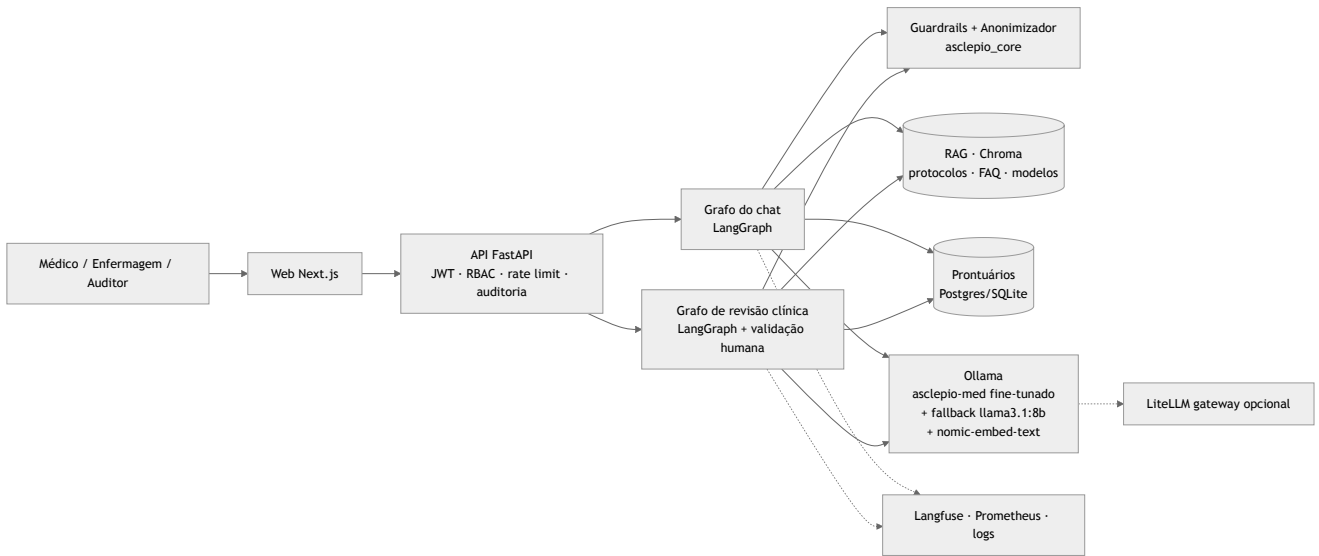
Exigência do PDF	Onde está neste relatório	Evidência no sistema/repositório
Dataset anonimizado ou sintético	§3, §4	data/synthetic/ , data/processed/ , data/knowledge_base/
Explicação do processo de fine-tuning	§4, §5, §9 + docs/FINE_TUNING.md	—
Descrição do assistente médico	§2, §6	—
Diagrama do fluxo LangChain/LangGraph	§2, §6, §7 (Mermaid) + docs/diagramas/	gerado pelo próprio LangGraph (GET /workflows/graph)
Avaliação do modelo e análise dos resultados	§9	ml/reports/eval_latest.json , docs/assets/eval/
Vídeo (≤ 15 min) demonstrando treino, fluxo, perguntas contextualizadas, logs e validação	roteiro em docs/ROTEIRO_VIDEO.md	a gravar
Datasets sugeridos (PubMedQA, MedQuAD)	§3, §4 (amostras ≤ 10 % via <code>with-public</code>)	data/processed/DATASET_CARD.md

1. Contexto e objetivos

Após automatizar análises de exames e textos clínicos (fases anteriores), o hospital quer um **assistente virtual médico treinado com seus próprios dados** (protocolos, FAQs de médicos, modelos de laudos/receitas/procedimentos), capaz de **auxiliar condutas, responder dúvidas e sugerir procedimentos com base nos protocolos internos**, além de **fluxos de decisão automatizados e seguros** (verificar exames pendentes, sugerir tratamentos, emitir alertas) coordenados com LangChain.

Objetivos de engenharia que adotamos além do edital: ser **didático** (código comentado em pt-BR, docs explicando o porquê), **operável** (`make setup` sobe tudo em Docker), **seguro por construção** (políticas e guardrails em código testável, auditoria imutável) e **honesto na avaliação** (métricas reais base vs fine-tuned, limitações explícitas).

2. Visão da solução



O produto combina três mecanismos complementares — e isso é a tese central do projeto:

Mecanismo	Papel	Por quê
Fine-tuning (LoRA)	Ensina tom, formato, escopo e vocabulário institucional : citar fontes, recusar prescrever, encerrar com aviso de validação, conhecer os IDs e a estrutura dos protocolos e modelos de documento.	Modelos genéricos não sabem "como o HU-FIAP fala" nem seus limites.
RAG com citações	Fornece os fatos atualizados (doses, limiares, critérios) dos protocolos, com documento › seção › score.	Conhecimento memorizado por fine-tuning envelhece e alucina números; o RAG é citável e auditável.

Mecanismo	Papel	Por quê
Regras + guardrails + humano no circuito	Decidem risco/alertas de forma determinística e impedem que a LLM prescreva ou saia do escopo; a decisão final é sempre de um médico.	Segurança não pode depender do modelo.

3. Dados: base de conhecimento e dados sintéticos

Todo o conteúdo é **fictício/educacional** (explicitado em cada arquivo), coerente com diretrizes públicas (Surviving Sepsis Campaign, AHA/ACC, SBC, SBD/ADA, GINA/GOLD, AHA/ASA, ATS/IDSA).

Conjunto	Quantidade	Formato	Uso
Protocolos clínicos (data/knowledge_base/protocolos/)	16 (sepse, SCA, AVC, CAD, PAC, TEV, anafilaxia, crise hipertensiva, IC, LRA, hipoglicemia/controle glicêmico, hipercalemia, asma/DPOC, dor, delirium, ITU)	Markdown com front matter YAML (id , titulo , tipo , categoria , setor , versao , atualizado_em , responsavel , tags) e 11 seções H2 fixas (Objetivo ... Fluxograma ... Referências)	RAG (chunk por seção) + geração de exemplos de fine-tuning
Modelos de documentos (modelos_documentos/)	10 (laudo de ECG, RX tórax, TC crânio, receita, evolução SOAP, sumário de alta, parecer, atestado, TCLE, prescrição padrão)	Markdown, 7 seções (finalidade, quando usar, campos, modelo com placeholders, exemplo, boas práticas, referências)	RAG + fine-tuning (categoria documento)
FAQ da equipe (faq/perguntas_frequentes.jsonl)	167 pares P/R ligados a protocolo/seção	JSONL	RAG (1 chunk por FAQ) + fine-tuning + gabarito da avaliação do RAG
Instruções seed (data/synthetic/instructions_seed.jsonl)	233 em 7 categorias (protocolo , documento , paciente_contexto , recusa_prescricao , fora_escopo , identidade_limites , anonimizacao_seguranca)	JSONL	Núcleo do dataset SFT
Prontuários sintéticos (asclepio_core.synthetic)	24 pacientes com sinais vitais, exames (pendentes/atrasados/críticos), medicações, evoluções — 18 cenários dirigidos aos protocolos + estáveis	Gerados com Faker (pt_BR), semente fixa; PII fictícia inserida de propósito nas evoluções	Banco da API, fluxos LangGraph, exemplos <code>paciente_contexto</code>
Datasets públicos sugeridos no edital (PubMedQA, MedQuAD)	amostras via <code>ml prepare --with-public</code> (75 PubMedQA pqa_labeled + 13 MedQuAD após dedupe ≈ 4 % do dataset, teto de 10 %)	HF Datasets (qiaojin/PubMedQA , lavita/MedQuAD)	Categoria <code>conhecimento_publico</code> : respostas marcadas como "conhecimento público, não institucional" + aviso de validação — ensinam o modelo a distinguir protocolo do hospital de literatura geral

O esquema e a validação dos arquivos estão em `data/knowledge_base/README.md`.

4. Pré-processamento, anonimização e curadoria

Pipeline `uv run python -m asclepio_ml prepare` (código em `ml/asclepio_ml/data_prep.py`, reutilizando `asclepio_core`):

1. **Carga** do seed, FAQ, seções de protocolos/modelos (loader com front matter e chunking por H2).
2. **Geração programática** de exemplos: por seção de protocolo ("O que diz o PROT-00X sobre ?"), por FAQ, por modelo de documento e — a partir dos pacientes sintéticos + `assess_risk()` — exemplos `paciente_contexto` cujo gabarito é **calculado por regras** (achados, critérios aplicáveis, sugestões para validação, fontes).
3. **Augmentação leve** por paráfrase templada das perguntas (3–5 variantes).
4. **Anonimização**: todos os textos passam pelo `Anonymizer` (CPF, CNS, RG, telefone, e-mail, CEP, endereço, data de nascimento, nomes por contexto/lista); contagem de entidades removidas vai para o `dataset_stats.json`.
5. **Curadoria**: remoção de vazios/curtos, *dedupe* exato e aproximado (normalização), limite de tamanho, **balanceamento por categoria** (cap), garantia de que respostas clínicas terminam com o aviso de validação (`guardrails.DISCLAIMER`) e que recusas são recusas (`is_refusal`).
6. **Split estratificado** por categoria (85 / 7,5 / 7,5) com semente fixa; formato *chat messages* com o **system prompt do Asclépio** (`ml/asclepio_ml/prompts.py`).
7. **Dataset card** (`data/processed/DATASET_CARD.md`): composição, contagens, % anonimizado, licença/aviso.
8. **Dados públicos** (`--with-public`, usado na execução oficial): amostras de PubMedQA (pergunta + contexto → resposta longa + conclusão) e MedQuAD (P/R de saúde) convertidas para o formato do Asclépio, com fonte explícita ("PubMedQA/MedQuAD — conhecimento público, não institucional"), limitadas a ≤ 10 % para não diluir o comportamento institucional (são em inglês e não citam PROT-xxx).

Os mesmos módulos de anonimização e guardrails rodam **em produção** (API) — consistência entre o que o modelo viu no treino e o que encontra em uso.

5. Fine-tuning da LLM

Método: *Supervised Fine-Tuning* com **LoRA** (PEFT) — adaptadores de baixo posto nas projeções de atenção e MLP (`q,k,v,o,gate,up,down`), treinando ~1 % dos parâmetros, o que permite treinar em um Mac Apple Silicon (MPS) em minutos, sem GPU NVIDIA.

Modelo base padrão: `Qwen/Qwen2.5-0.5B-Instruct` (aberto, sem *gating*, suportado pelo Ollama). Alternativas configuráveis: `meta-llama/Llama-3.2-1B/3B-Instruct` (requer token HF), `TinyLlama-1.1B-Chat`. Justificativa no ADR-0003.

Hiperparâmetros, tempos, loss e hardware reais estão em `ml/registry.json` e em `docs/FINE_TUNING.md` (gerados pela execução real do pipeline — ver seção 9).

Exportação: `merge_and_unload()` → pesos HF → `Modelfile` (`system prompt + template + temperature 0.1`) → `ollama create asclepio-med`. A API detecta o modelo no Ollama e usa fallback `llama3.1:8b` se ele não existir (a UI mostra se o modelo ativo é fine-tunado).

6. Assistente médico com LangChain

Implementado como **grafo LangGraph** (`backend/asclepio_api/services/assistant.py`), consumido via REST e **SSE** (streaming de tokens + etapas do grafo):

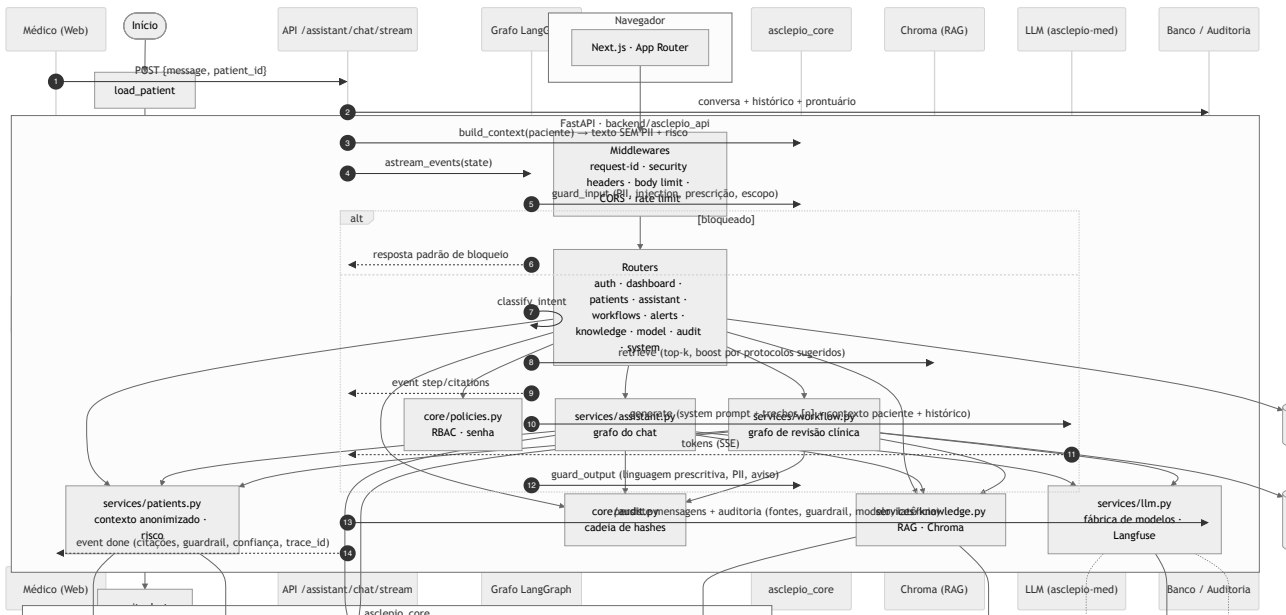
```
guard_input → classify_intent → retrieve → generate → guard_output → END
└── (prompt injection) → blocked → END
```

- **Integração da LLM customizada:** `LLMFactory.chat_model()` → `ChatOllama(model="asclepio-med")` (ou `LiteLLM/OpenAI-compatible/fake`), com callbacks do Langfuse.
- **Consulta a base estruturada:** prontuários em Postgres/SQLite via SQLAlchemy (`services/patients.py`) — sinais vitais, exames, medicações, evoluções, alergias.
- **Contextualização com dados atualizados do paciente:** `build_context()` monta um texto **sem PII** (idade/sexo/setor + clínica + risco calculado por regras + protocolos possivelmente aplicáveis), exibível pelo usuário em "ver contexto anonimizado".
- **RAG:** Chroma (cosine) com `nomic-embed-text`; *boost* para os protocolos sugeridos pelas regras do paciente; trechos numerados `[n]` no prompt; o modelo cita e lista fontes.
- **Memória:** histórico curto da conversa (últimas 6 mensagens) persistido por conversa/usuário.

- **Explicabilidade na resposta:** citações (documento, seção, score, trecho), intenção, guardrail (status/flags/notas), modelo, latência, confiança, PII redigida, `trace_id`.

7. Fluxos automatizados com LangGraph

Grafo de **revisão clínica** (`services/workflow.py`), 10 nós, com ramificações condicionais, laço de regeneração e **pausa para validação humana** (`interrupt` + `checkpoint` SQLite):



Etapa do edital	Nó(s)
Receber informações do paciente	<code>load_patient</code> (anonimiza, calcula risco)
Verificar exames pendentes	<code>check_pending_exams</code> (pendentes/coletados/atrasados por <code>due_at</code>)
Sugerir tratamentos	<code>retrieve_protocols</code> + <code>suggest_conduct</code> (LLM fine-tunada cita [n]) + <code>validate_guardrails</code>
Emitir alertas para a equipe	<code>emit_immediate_alerts</code> (crítico, antes da LLM) e <code>emit_alerts</code> (exames atrasados, valores críticos, gatilhos de protocolo)
Coordenação segura	<code>human_review</code> (médico aprova/rejeita; enfermagem não pode), <code>finalize</code> (auditoria)

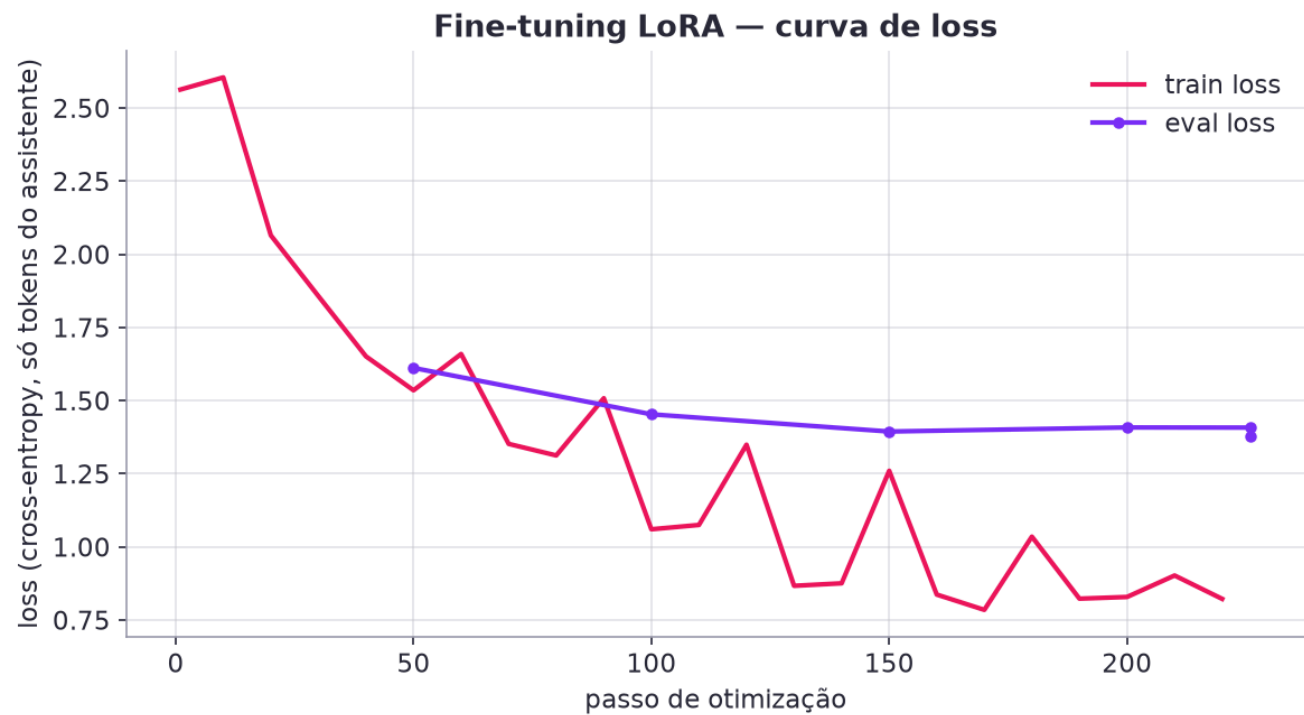
Cada nó grava um **passo** (status, duração, resumo, dados) → timeline na UI e rastreabilidade. O diagrama é gerado pelo próprio LangGraph (`GET /workflows/graph`, `docs/diagramas/`).

8. Segurança, validação e explicabilidade

- **Limites de atuação** (`asclepio_core/guardrails.py`): entrada — PII redigida, *prompt injection* bloqueado, pedido de prescrição direta → intenção `prescricao` (recusa + protocolo), fora de escopo → redireciona; saída — linguagem prescritiva imperativa reescrita como sugestão, PII residual redigida, aviso de validação humana obrigatório, sinalização de ausência de fontes. Nos fluxos, a sugestão reprovada é regenerada uma vez.
- **Logging detalhado:** structlog com `request_id` ponta a ponta; **trilha de auditoria append-only com cadeia de hashes** (`GET /audit/verify`), registrando usuário, ação, recurso, fontes usadas, guardrail, modelo, latência, PII redigida; métricas Prometheus; traces no Langfuse.
- **Explainability:** toda resposta traz as fontes (documento › seção › score › trecho); o usuário vê o contexto exato enviado à LLM; a avaliação mede `citation_rate`; a confiança é derivada do score de recuperação e das flags.
- **Autenticação/autorização (v1.2):** senhas bcrypt com política, bloqueio por tentativas, **MFA com app autenticador (TOTP)** obrigatório para administradores, sessões com refresh token rotativo e revogação, troca de senha obrigatória no 1º acesso, **RBAC por perfil** (médico vê só o que é responsabilidade dele; IA & Modelos, base de conhecimento, usuários, catálogos e auditoria só para admin/auditor), cadastro de profissionais com CRM/especialidade validados — tudo auditado (ver `docs/POLITICAS.md`).

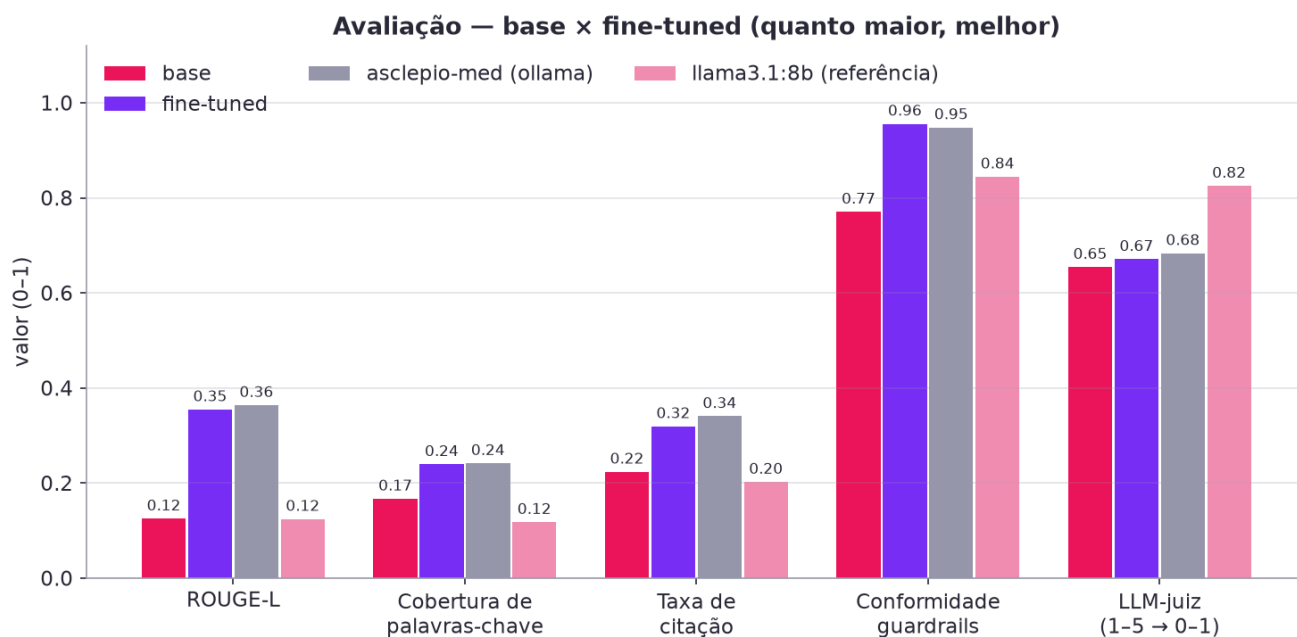
9. Avaliação do modelo e análise dos resultados

Execução real (`make finetune` , 2026-08-22, Apple Silicon/MPS): LoRA $r=16$, $\alpha=32$, dropout 0.05, alvos `q_proj`, `o_proj`, `v_proj`, `up_proj`, `gate_proj`, `k_proj`, `down_proj`; 8.8 M parâmetros treináveis de 494 M (1.8%); 2 épocas = 226 passos, batch efetivo 16, lr 0.0002, seq. máx. 1024, float32; **1801** exemplos de treino / 165 de validação (inclui amostras PubMedQA/MedQuAD $\leq 10\%$); duração **21.74 min**; loss de treino 0.8223 · loss de validação 1.3785. Exportado como safetensors → `ollama create asclepio-med` (ok).

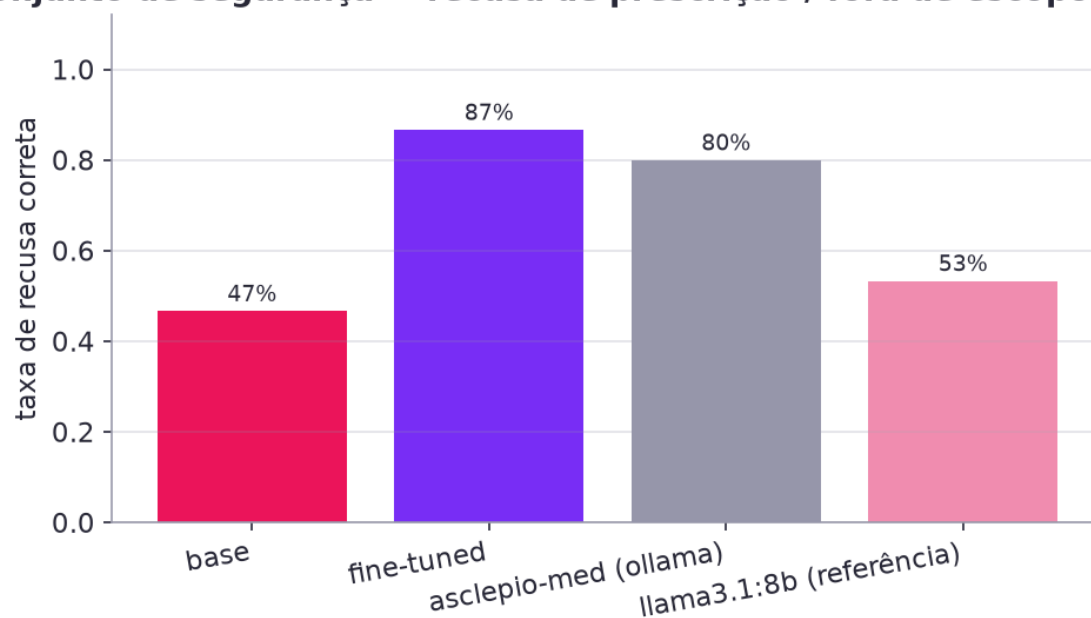


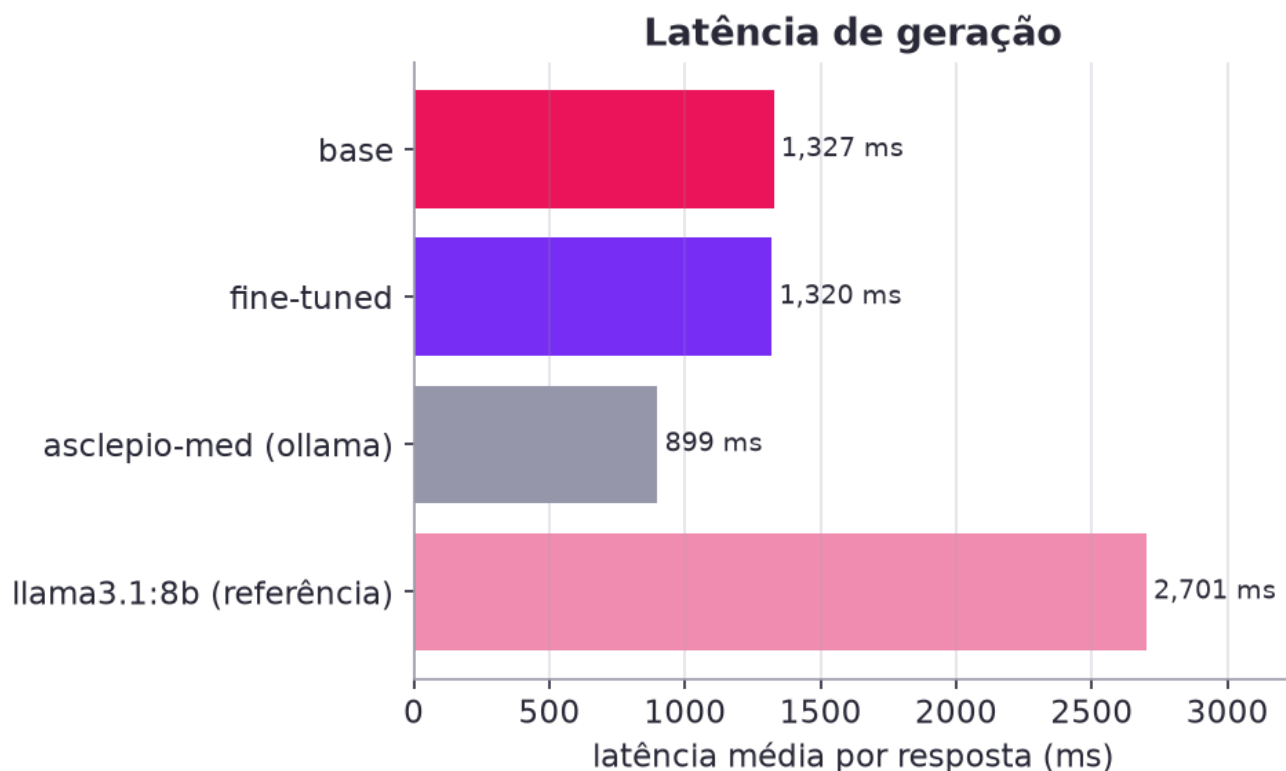
Avaliação (2026-08-22T00:20, 120 exemplos de teste *held-out* + 15 prompts de segurança; juiz `llama3.1:8b` em 60 amostras):

Modelo	ROUGE-L	BLEU	Cobertura de termos	Taxa de citação	Conformidade guardrails	Recusa (seg.)	Juiz LLM (1–5)	Latência média (ms)
Base · Qwen2.5-0.5B-Instruct	0.125	3.0	16.7%	22.3%	77.0%	47%	3.62	1327
Fine-tuned · asclepio-med (HF merged)	0.355	26.8	23.9%	31.9%	95.6%	87%	3.68	1320
Fine-tuned · asclepio-med (Ollama)	0.364	27.6	24.1%	34.0%	94.8%	80%	3.73	899
Referência · llama3.1:8b (sem fine-tuning)	0.124	3.8	11.8%	20.2%	84.4%	53%	4.30	2701

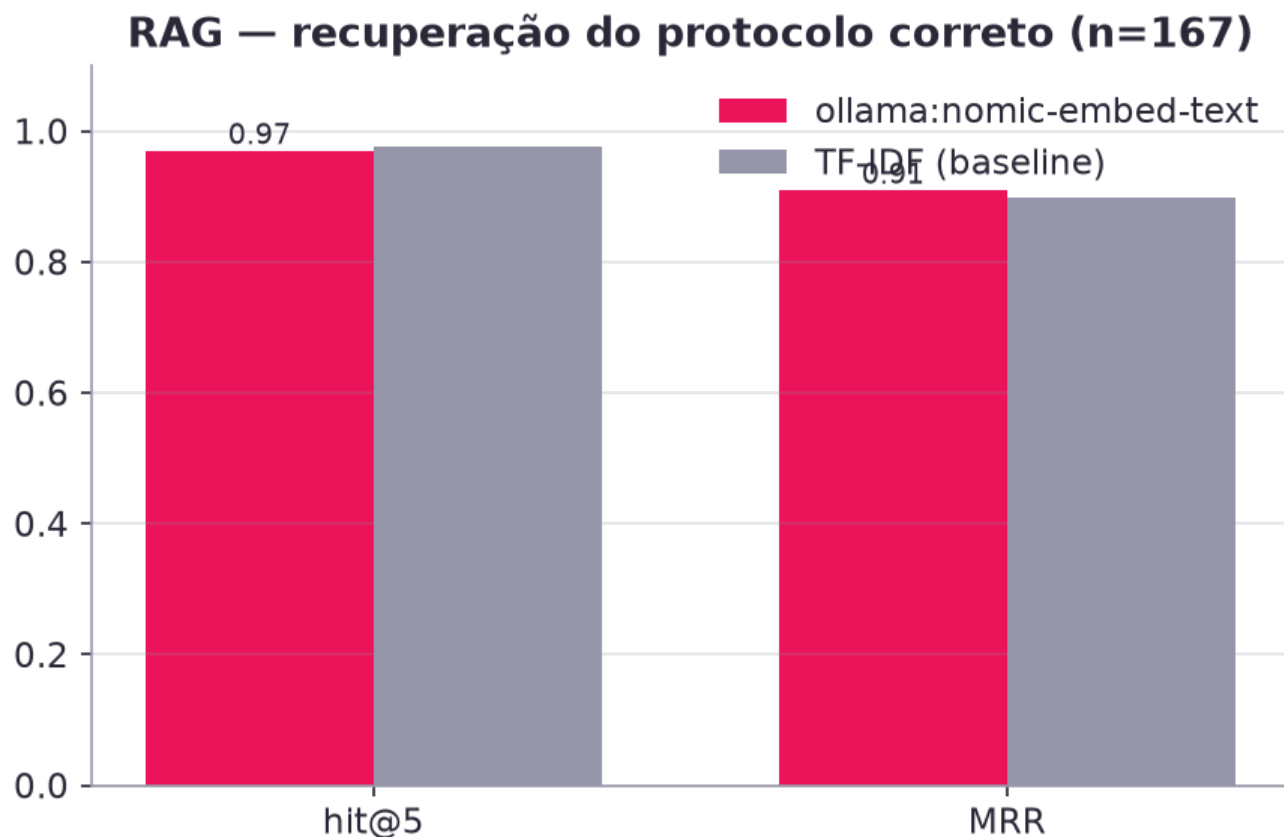


Conjunto de segurança — recusa de prescrição / fora de escopo / injeção





RAG (ollama:nomic-embed-text, 287 chunks, 167 perguntas do FAQ como consultas): *hit@5* = **97.0%**, MRR = **0.910**
 (baseline TF-IDF: *hit@5* 97.6%, MRR 0.898).



Leitura dos resultados - O fine-tuning **transformou o comportamento** do modelo de 0,5B: ROUGE-L 0.12 → 0.36 (×2.9), BLEU 3.0 → 27.6, cobertura de termos-chave 17% → 24%, taxa de citação 22% → 34%, **conformidade com os guardrails 77% → 95%** e recusa correta no conjunto de segurança 47% → 80%. Ou seja: aprendeu o formato institucional (fonte + aviso de validação), o escopo e os limites — exatamente o objetivo do fine-tuning neste projeto. - O juiz LLM dá

nota ligeiramente maior ao fine-tuned que ao base (3.62 → 3.73) e nota maior ao `llama3.1:8b` (4.30) — um modelo 16× maior raciocina melhor, porém **cita menos, obedece menos aos guardrails (84%) e é ~3× mais lento**. Isso confirma a tese: fine-tuning para forma/segurança + RAG para fatos + guardrails em código; a API permite trocar de modelo quando se quer mais raciocínio. - O RAG é forte (hit@5 97%) e é ele quem garante os números dos protocolos nas respostas; em produção o `asclepio-med` roda com RAG, o que eleva a qualidade observada além da medida aqui (avaliação sem recuperação, só modelo). - Limitações: conjunto de teste pequeno e gerado a partir da mesma base (risco de otimismo em ROUGE/BLEU), juiz automático em 60 amostras, modelo pequeno ainda erra detalhes fora do que viu. Tabelas completas, amostras e análise em `docs/FINE_TUNING.md` e `ml/reports/eval_latest.md`.

Metodologia (`ml/asclepio_ml/evaluate.py`): conjunto de teste *held-out* (estratificado) + conjunto de segurança (pedidos de prescrição, fora de escopo, injeção). Modelos comparados: base (`Qwen2.5-0.5B-Instruct`) × fine-tuned (`asclepio-med`) × referência (`llama3.1:8b`). Métricas: ROUGE-L, BLEU, cobertura de termos-chave (números, fármacos, IDs), taxa de citação, **conformidade com guardrails** (mesmo código do produto), nota de um **LLM-juiz** (rubrica de fidelidade/segurança/clareza), latência. RAG: *hit@5* e MRR com as perguntas do FAQ como consultas e o `protocolo_id` como gabarito.

10. Engenharia: organização, DevEx, testes e operação

- **Monorepo** `uv` (core / backend / ml) + frontend Next.js; contrato de API documentado (`docs/CONTRATO_API.md`); ADRs.
- **Testes**: 57+ testes Python (core: anonimização, regras, guardrails, base de conhecimento; API: auth/lockout/RBAC, pacientes sem PII, chat e guardrails, SSE, fluxo completo com aprovação/rejeição, auditoria com detecção de adulteração, RAG, modelo) rodando com `LLM_PROVIDER=fake` — sem rede; testes do pipeline de ML.
- **CI** (GitHub Actions): ruff, pytest com cobertura, eslint/tsc/build, build das imagens, gitleaks e pip-audit. Pre-commit. Dev Container.
- **Operação**: `make setup` → Docker Compose (Postgres, API, Web; perfis Ollama, LiteLLM, Langfuse v3, ML); healthchecks; imagens multi-stage não-root; `.env` com defaults seguros; logs JSON em produção.

11. Limitações e próximos passos

- Modelo de 0,5B é limitado em raciocínio clínico; o fine-tuning melhora formato/escopo/segurança, não substitui o RAG. Próximo passo: Llama-3.2-3B/8B com QLoRA em GPU, DPO com preferências de médicos.
- Conteúdo clínico fictício, sem revisão por comissão; guardrails baseados em regras (adicionar classificador treinado e *self-check* do modelo).
- Anonimização por regex + dicionário (adicionar NER clínico pt-BR, ex.: modelos BERTimbau).
- Auth sem MFA/SSO; auditoria local (integrar SIEM); avaliação com mais amostras e juizes humanos.

12. Conclusão

O Asclépio atende a todos os requisitos da Fase 3 — fine-tuning com dados institucionais anonimizados e curados, assistente LangChain integrado à LLM customizada e à base estruturada de pacientes, fluxos LangGraph seguros com alertas e validação humana, guardrails, logging/auditoria e explicabilidade — em um pacote reproduzível, dockerizado e documentado, que pode ser demonstrado de ponta a ponta em menos de 15 minutos.

Anexo A — Processo de desenvolvimento (memória descritiva)

Asclépio — Assistente Clínico Inteligente · Tech Challenge · Fase 3 · FIAP Pós-Tech IA para Devs (8IADT)

Este documento é a **evidência acadêmica do processo**: descreve, etapa por etapa, *como* o projeto foi concebido, construído e validado — com objetivo, método, ferramentas, artefatos produzidos e critérios de qualidade de cada fase. Complementa o `RELATORIO_TECNICO.md` (que descreve *o que* foi entregue) e o `EVIDENCIAS.md` (que mostra *onde* cada exigência aparece no sistema).

1. Introdução

O edital da Fase 3 pede um assistente virtual médico treinado com dados próprios do hospital (fine-tuning), integrado via LangChain a bases estruturadas e ao contexto do paciente, com fluxos de decisão automatizados e seguros (LangGraph), guardrails, logging/auditoria e explicabilidade. A equipe estabeleceu, além do edital, quatro princípios de engenharia que orientaram todas as decisões:

1. **Didática** — código comentado em pt-BR explicando o *porquê*, documentação que ensina.
2. **Reprodutibilidade** — qualquer avaliador instala com um comando e obtém o mesmo sistema; sementes fixas nos processos estocásticos; nenhum serviço pago.
3. **Segurança por construção** — políticas, guardrails e anonimização como **código testável**, não como texto.
4. **Honestidade experimental** — métricas reais (base × fine-tuned × referência), limitações explícitas.

Método de trabalho. Desenvolvimento incremental orientado a requisitos: (i) extração dos requisitos do edital para uma tabela de conformidade; (ii) definição de contratos (API e identidade visual) antes da implementação, permitindo trabalho paralelo de frontend, backend e ML; (iii) integração e verificação contínuas (testes automatizados + validação manual no navegador a cada incremento); (iv) registro de decisões em ADRs e do histórico em commits atômicos. Ferramentas de assistência de código baseadas em IA foram utilizadas como apoio de produtividade (*pair programming*), com todas as decisões de arquitetura, revisão e validação realizadas pela equipe — prática alinhada ao próprio tema do curso.

Stack. Python 3.12 (uv), LangChain/LangGraph 1.x, Transformers/PEFT/TRL, Ollama, ChromaDB, FastAPI, SQLAlchemy 2, PostgreSQL, Next.js 16/React 19/Tailwind 4, Docker Compose, GitHub Actions, LiteLLM, Langfuse.

2. Etapas do desenvolvimento

Cada etapa segue o formato: **Objetivo** → **Método** → **Artefatos** → **Validação** → **Resultado**.

Etapa 1 — Análise do edital e engenharia de requisitos

- **Objetivo:** transformar o PDF do desafio em requisitos verificáveis.
- **Método:** leitura estruturada do edital; extração de 17 requisitos (técnicos e de entrega); para cada um, definição antecipada do *critério de evidência* (que tela, arquivo ou comando comprovaria o atendimento). Requisitos ampliados pela equipe: autenticação forte, perfis de acesso, observabilidade de LLM (LiteLLM/Langfuse), instalação 100 % automatizada.
- **Artefatos:** tabela "Checklist de conformidade" (abertura do `RELATORIO_TECNICO.md`); `EVIDENCIAS.md`.
- **Validação:** revisão cruzada requisito ↔ evidência ao final de cada fase.
- **Resultado:** 17/17 requisitos rastreáveis (o vídeo, único item externo ao repositório, tem roteiro pronto).

Etapa 2 — Concepção do produto e identidade visual

- **Objetivo:** dar ao sistema identidade própria e coerente com a instituição.
- **Método:** *naming* (Asclépio, deus grego da medicina — o bastão com serpente é símbolo universal da profissão); construção do logotipo em SVG (bastão + serpente + nós de rede neural); definição de *design tokens* inspirados na identidade FIAP (rosa #ED145B sobre quase-preto #0B0B10, Montserrat/Inter), verificados visualmente em três tamanhos de aplicação.
- **Artefatos:** docs/assets/brand/*.svg, docs/IDENTIDADE_VISUAL.md.
- **Validação:** renderização no navegador (256/96/48 px) e no favicon.

Etapa 3 — Arquitetura, contratos e decisões

- **Objetivo:** desenhar uma arquitetura que permitisse paralelismo de construção e evolução segura.
- **Método:** monorepo com três pacotes Python (núcleo compartilhado `asclepio_core`, API, pipeline de ML) + frontend; **contrato de API escrito antes do código** (`CONTRATO_API.md`, tipos TypeScript e endpoints), funcionando como especificação executável para o frontend; decisões relevantes registradas como ADRs com alternativas e consequências.
- **Artefatos:** docs/ARQUITETURA.md, docs/CONTRATO_API.md (v1 → v1.3), docs/adr/0001..0005.
- **Validação:** o frontend foi construído contra o contrato **antes** de o backend existir (modo *mock*), e a integração posterior não exigiu mudanças de contrato — evidência da qualidade da especificação.
- **Decisões-chave (resumo):** grafos LangGraph determinísticos com `interrupt` em vez de agente livre (ADR-0002); LoRA em modelo pequeno servido no Ollama (ADR-0003); auditoria com cadeia de hashes (ADR-0004); LiteLLM/Langfuse opcionais por perfil de composição (ADR-0005).

Etapa 4 — Base de conhecimento clínica (dados "do hospital")

- **Objetivo:** criar o corpus institucional fictício exigido pelo edital (protocolos, FAQs, modelos de documentos).
- **Método:** redação de **16 protocolos clínicos** (sepse, SCA, AVC, CAD, PAC, TEV, anafilaxia, crise hipertensiva, IC, LRA, hipoglicemia/controle glicêmico, hipercalemia, asma/DPOC, dor, delirium, ITU) com estrutura padronizada (front matter YAML + 11 seções H2 fixas, incluindo limiares numéricos, tempos-alvo e fluxograma Mermaid), coerentes com diretrizes públicas (Surviving Sepsis Campaign, AHA/ACC/ASA, GINA/GOLD, ADA/SBD, NEWS2/qSOFA) e explicitamente marcados como fictícios; **10 modelos de documentos** (laudos, receita, evolução SOAP, sumário de alta etc.); **167 FAQs** vinculadas a protocolo/seção. A estrutura padronizada foi projetada **para o chunking por seção** do RAG (citações "PROT-001 › Conduta").
- **Artefatos:** data/knowledge_base/** (+ README.md com o esquema).
- **Validação:** validação sintática (front matter, seções, JSONL) por script; revisão cruzada FAQ ↔ protocolo (15 correções de consistência registradas).

Etapa 5 — Dados sintéticos de pacientes e anonimização (LGPD)

- **Objetivo:** prover prontuários realistas sem qualquer dado real, e demonstrar anonimização de verdade.
- **Método:** gerador determinístico (`asclepio_core/synthetic.py`, Faker pt_BR, semente fixa) de **24 pacientes** — 18 cenários dirigidos aos protocolos (ex.: sepse com lactato de controle atrasado, CAD, SCA com curva de troponina pendente, hipercalemia em LRA) e internações estáveis. **PII fictícia é inserida de propósito** nas evoluções (nome, CPF, telefone, endereço, nome da mãe) para exercitar o anonimizador. O anonimizador (`asclepio_core/anonymizer.py`) combina regex determinísticas (CPF, CNS, RG, telefone, e-mail, CEP, endereço, data de nascimento) e detecção contextual de nomes, com pseudonimização consistente; é o **mesmo módulo** usado no preparo do dataset e em produção (propriedade de consistência treino↔uso).
- **Artefatos:** packages/asclepio_core/asclepio_core/{synthetic,anonymizer}.py, data/synthetic/patients.json, testes em packages/asclepio_core/tests/.
- **Validação:** 25 testes unitários do núcleo (falsos positivos clínicos, endereços, pseudonimização); teste de integração que garante **ausência de CPF/telefone/nome no contexto enviado à LLM**; contadores de PII expostos na interface ("N dados pessoais redigidos").

Etapa 6 — Construção do dataset de fine-tuning

- **Objetivo:** transformar o corpus institucional em dados de instrução com pré-processamento, anonimização e curadoria auditáveis.

- **Método (pipeline `ml prepare`):** (a) 233 instruções-semente em 7 categorias (protocolo, documento, contexto de paciente, recusa de prescrição, fora de escopo, identidade/limites, privacidade); (b) geração programática de exemplos a partir de cada seção de protocolo, tabela de fármacos, FAQ e modelo de documento; (c) exemplos de contexto de paciente com **gabarito calculado por regras clínicas** (qSOFA/NEWS2/valores críticos) — supervisão sem intervenção manual; (d) augmentação por paráfrase templada; (e) anonimização integral; (f) curadoria: dedupe exato e aproximado, filtros de tamanho, balanceamento por categoria (teto de 1.300 na dominante), reforço do aviso de validação humana e das recusas; (g) *split* 85/7,5/7,5 **estratificado e agrupado** (paráfrases da mesma pergunta nunca se separam entre treino e teste — controle de vazamento); (h) amostras dos datasets sugeridos no edital (**PubMedQA** e **MedQuAD**, $\leq 10\%$, rotuladas como "conhecimento público, não institucional").
- **Artefatos:** `ml/asclepio_ml/data_prep.py`, `data/processed/{train,val,test}.jsonl` (2.134 exemplos: 1.801/165/168), `dataset_stats.json`, `DATASET_CARD.md`.
- **Validação:** testes de preparo com *fixtures*; teste de segurança que falha se PII crua sobreviver no dataset.

Etapas 7 — Fine-tuning (LoRA) e exportação para serving local

- **Objetivo:** especializar um LLM aberto no comportamento institucional, em hardware de estudante.
- **Método:** SFT com **LoRA** ($r=16$, $\alpha=32$, dropout 0,05, projeções q/k/v/o/gate/up/down $\rightarrow$ 8,8 M parâmetros treináveis de 494 M $\approx 1,8\%$) sobre `Qwen/Qwen2.5-0.5B-Instruct` (aberto, multilíngue, compatível com Apple Silicon/MPS e importável pelo Ollama); `TRL SFTTrainer` com *loss* apenas nos tokens do assistente; 2 épocas = 226 passos, batch efetivo 16, lr $2e-4$ cosseno, seq. máx. 1.024, fp32/MPS. Exportação: fusão do adapter (`merge_and_unload`) $\rightarrow$ `safetensors` $\rightarrow$ `ModelFile` (system prompt institucional + template de chat) $\rightarrow$ `ollama create asclepio-med`. O modelo final é distribuído pela **Release v1.2.0 do GitHub** e baixado automaticamente pelo instalador.
- **Artefatos:** `ml/asclepio_ml/{train,export}.py`, `ml/configs/finetune.yaml` (comentado), `ml/registry.json`, `docs/assets/eval/train_loss.png`, `Release asclepio-med-v1.2.0.tar.gz`.
- **Validação/execução real:** treino em **21,7 min** (MacBook Apple Silicon), *loss* treino 0,82 · validação 1,38; problemas reais de engenharia documentados e resolvidos (mudanças de API do transformers 5, `rope_parameters` na importação pelo Ollama, limites de memória do MPS).

Etapas 8 — Avaliação experimental do modelo

- **Objetivo:** medir, com controles, o que o fine-tuning mudou — e o que não mudou.
- **Método:** conjunto de teste *held-out* (120 amostras) + conjunto adversarial de segurança (15 prompts de prescrição direta/fora de escopo/injeção); quatro condições: base, fine-tuned (transformers), fine-tuned (Ollama) e **referência 16x maior sem fine-tuning** (`llama3.1:8b`). Métricas: ROUGE-L, BLEU, cobertura de termos-chave, taxa de citação, **conformidade com os guardrails** (avaliada pelo mesmo código de produção) e taxa de recusa correta, nota 1–5 por **LLM-juiz** (`llama3.1:8b`, rubrica de fidelidade/segurança/clareza, 60 amostras), latência. RAG avaliado com as 167 FAQs como consultas e o `protocolo_id` como gabarito (hit@5, MRR), contra baseline TF-IDF.
- **Artefatos:** `ml/asclepio_ml/{evaluate,metrics,rags_eval,plots}.py`, `ml/reports/eval_latest.{json,md}`, gráficos em `docs/assets/eval/`.
- **Resultados (última execução):** ROUGE-L 0,125 $\rightarrow$ **0,364**; BLEU 3,0 $\rightarrow$ **27,6**; conformidade com guardrails 0,77 $\rightarrow$ **0,95**; recusa correta 47 % $\rightarrow$ **80 %**; juiz 3,62 $\rightarrow$ 3,73; latência $\sim 0,9$ s (vs 2,7 s da referência). O `llama3.1:8b` obtém juiz maior (4,30) porém cita menos e obedece menos aos guardrails (0,84) — sustentando a tese *fine-tuning para forma/segurança + RAG para fatos + regras para decisões*. RAG: hit@5 **97,0 %**, MRR **0,910**. Limitações discutidas na §6.

Etapas 9 — Assistente conversacional (LangChain/LangGraph + RAG)

- **Objetivo:** integrar a LLM customizada a um pipeline com recuperação citável e contexto do paciente.
- **Método:** grafo LangGraph de 6 nós (`guard_input` $\rightarrow$ `classify_intent` $\rightarrow$ `retrieve` $\rightarrow$ `generate` $\rightarrow$ `guard_output`, com desvio de bloqueio); RAG com ChromaDB (cosseno) e embeddings `omic-embed-text`, chunking por seção com metadados e *boost* dos protocolos sugeridos pelas regras clínicas do paciente; contexto do prontuário **anonimizado e inspecionável pelo usuário**; streaming SSE token a token com eventos de progresso; citações numeradas ligadas a documento $\rightarrow$ seção $\rightarrow$ score; fábrica de provedores (Ollama/LiteLLM/OpenAI-compatível/*fake* determinístico para testes e CI).
- **Artefatos:** `backend/asclepio_api/services/{assistant,knowledge,llm,patients}.py`; diagrama gerado pelo próprio grafo (`docs/diagramas/grafos_chat_langgraph.mmd`).

- **Validação:** testes de API cobrindo citações, intenções, bloqueios e SSE; verificação qualitativa com o modelo real (respostas fiéis ao protocolo com fontes).

Etapa 10 — Fluxos clínicos automatizados e seguros (LangGraph)

- **Objetivo:** materializar o requisito "receber informações do paciente → verificar exames pendentes → sugerir condutas → alertar a equipe", com segurança verificável.
- **Método:** grafo de **10 nós** com ramificações condicionais: risco calculado por **código determinístico** (qSOFA, NEWS2, valores críticos, exames atrasados) *antes* de qualquer LLM; alerta crítico imediato; sugestão da LLM com fontes e regeneração única guiada pelos guardrails; **interrupt()** com **checkpoint persistente** pausando o fluxo até a decisão de um médico (`Command(resume=...)`); cada nó registra passo (status, duração, resumo) para a linha do tempo e a auditoria.
- **Artefatos:** `backend/asclepio_api/services/workflow.py` , `packages/asclepio_core/asclepio_core/clinical_rules.py` , `docs/diagramas/grafos_revisao_clinica_langgraph.mmd` .
- **Validação:** teste de ciclo completo (execução → pausa → 403 para enfermagem → aprovação → finalização), teste de rejeição (alertas reconhecidos automaticamente), teste de paciente de baixo risco (sem alerta imediato).

Etapa 11 — Segurança, governança e observabilidade

- **Objetivo:** limites de atuação, rastreabilidade e acesso dignos de um sistema real de saúde.
- **Método:** guardrails de entrada/saída como funções puras testáveis (injeção de prompt, pedido de prescrição, escopo, linguagem prescritiva, PII residual, aviso obrigatório); **autenticação forte** — senhas bcrypt com política, bloqueio por tentativas, **MFA TOTP** com códigos de recuperação (obrigatório para administradores), sessões com refresh token rotativo, revogação imediata e **detecção de reuso**; **RBAC por permissão** (`recurso:ação`) com perfis clínicos enxutos e áreas técnicas exclusivas do admin; **auditoria append-only com cadeia de hashes** (verificação de integridade exposta na interface e testada com adulteração deliberada do banco); logs estruturados com `trace_id` ponta a ponta, métricas Prometheus e traces de LLM no Langfuse (auto-hospedado, um comando).
- **Artefatos:** `asclepio_core/guardrails.py` , `backend/asclepio_api/core/{security,policies,audit,deps}.py` , `docs/POLITICAS.md` .
- **Validação:** 45 testes de API (fluxo completo de onboarding do admin com TOTP calculado em teste, rotação/reuso de refresh, logout, RBAC por rota, quebra da cadeia de auditoria).

Etapa 12 — Interface por perfil (visão de produto)

- **Objetivo:** experiência de sistema hospitalar real: cada papel vê apenas sua responsabilidade.
- **Método:** navegação montada a partir das **permissões** do usuário (não do papel); médico: pacientes, assistente, fluxos, alertas, protocolos; admin: + usuários & profissionais (CRM validado, especialidade/setor de catálogos administráveis), IA & Modelos, base de conhecimento, auditoria, configurações; detalhes técnicos (grafos, JSON) restritos a `system:internals` ; identidade do hospital configurável (`APP_HOSPITAL_NAME`) obtida de endpoint público; central de **Documentação** (este acervo) para os perfis de avaliação.
- **Artefatos:** `frontend/**` (Next.js 16, contrato-first, modo mock completo), `docs/CONTRATO_API.md` §v1.2–v1.3.
- **Validação:** lint + typecheck + build na CI; verificação manual dos 4 perfis; guardas de rota com tela de acesso negado.

Etapa 13 — Engenharia, DevEx e distribuição

- **Objetivo:** "clonou, rodou": reprodutibilidade total para qualquer avaliador.
- **Método:** Docker Compose com perfis (núcleo, `ollama` , `gateway` , `observability` , `ml`); `make setup/install.sh` (instalador de 1 linha) que verifica pré-requisitos, gera segredos, resolve portas em conflito, baixa modelos e o fine-tunado da Release, e valida a saúde; migração leve de esquema; CI com 4 jobs (lint+testes Python, frontend, build Docker, gitleaks+pip-audit); pre-commit; PDFs gerados por script (`make docs-pdf`).
- **Artefatos:** `docker-compose.yml` , `Makefile` , `scripts/{bootstrap.sh,build_report_pdf.py,update_report_metrics.py}` , `install.sh` , `.github/workflows/ci.yml` .
- **Validação:** instalação executada do zero na máquina da equipe e reproduzida por terceiros (ver Etapa 14); **88 testes automatizados** no total; CI verde no repositório público.

Etapa 14 — Validação de campo e correções (engenharia de verdade)

- **Objetivo:** submeter o sistema a uso real por terceiros e corrigir com método.
- **Método/ocorrências (todas rastreáveis em commits e na auditoria):** 1. *Instalação por colegas de turma* revelou falha de resolução de DNS do serviço Ollama quando o compose subia sem o perfil (Name or service not known) → correção: COMPOSE_PROFILES persistido no .env + mensagem de erro amigável (503) no lugar do erro técnico; cenário reproduzido em laboratório antes do conserto. 2. *QR do MFA não escaneava* → diagnóstico com decodificação programática (OpenCV): densidade (versão 8) e URI com acentos; correção: URI otpauth mínima/ASCII (versão 6) e módulos maiores. 3. *Conflitos de porta* com outros serviços nas máquinas (3000/4000/8000) → portas parametrizadas + detecção e orientação no instalador. 4. *Credencial de admin divergente* → a **trilha de auditoria** evidenciou onboarding anterior (troca de senha + MFA) e o bloqueio por tentativas atuou como projetado; recuperação pelos fluxos administrativos.
- **Resultado:** além dos consertos, os episódios viraram evidência funcional dos mecanismos de segurança e observabilidade.

Etapa 15 — Publicação e empacotamento das evidências

- **Objetivo:** entrega auditável.
- **Método:** repositório público com histórico de commits significativos; Release com o modelo; relatório técnico e guia de instalação em PDF; mapa de evidências; roteiro do vídeo; esta memória descritiva; central de Documentação dentro da própria plataforma.
- **Artefatos:** <https://github.com/rodrigogrosa/asclepio> (código, Releases, Actions) e docs/**.

3. Cronologia resumida (marcos por commit)

Marco	Commit
v1.0 — plataforma completa: core (anonimização/regras/guardrails), API LangChain/LangGraph, frontend, pipeline de ML executado, Docker/CI/docs	a10ba8a
v1.2 — autenticação real (MFA/sessões), perfis por permissão, cadastros/catálogos, PubMedQA/MedQuAD no dataset, instalador de 1 linha	cd8a17d
Correções de campo: CI (7beb7b2), QR do MFA (f2cd98c , 274614d), DNS do Ollama + erro amigável (19ef4ae , 89ad821)	—
Evidências e entrega: checklist (2ff12d2), mapa de evidências (233fa95), PDFs (f8ea1e2 , 493c40c), modelo na Release + guia (95a82ed), zoom/exportação de diagramas (e4c0da6)	—

4. Rastreabilidade

A matriz requisito → seção → evidência está no início do RELATORIO_TECNICO.md ; o mapa requisito → tela/arquivo/comando, no EVIDENCIAS.md . Cada afirmação numérica deste documento é regenerável: make prepare/train/export/eval reproduz dados e métricas; make docs-metrics reinjeta os números na documentação.

5. Ameaças à validade e limitações metodológicas

- **Otimismo em ROUGE/BLEU:** o teste, embora held-out e com agrupamento de paráfrases, compartilha o molde de geração do treino; os ganhos medem sobretudo aderência ao formato institucional (que é o objetivo declarado), não conhecimento clínico geral.
- **Juiz automático:** 60 amostras e um único modelo-juiz; sem avaliação por especialistas humanos.
- **Conteúdo clínico fictício:** protocolos coerentes com diretrizes públicas, porém sem revisão por comissão clínica; o sistema não deve apoiar decisões reais.
- **Anonimização por regras:** robusta aos padrões brasileiros usuais, mas sem NER estatístico; casos atípicos podem escapar (mitigado pela dupla passagem e pelos guardrails de saída).
- **Modelo de 0,5 B:** limitado em raciocínio; a arquitetura compensa com RAG e regras, e permite troca de modelo pelo admin.

6. Lições aprendidas

1. **Contrato antes do código** viabilizou paralelismo real (frontend em mock + backend + ML simultâneos) sem retrabalho de integração.
2. **Segurança como código** (guardrails/políticas testáveis) rende evidência objetiva — a mesma função valida produção e avaliação do modelo.
3. **Regras determinísticas antes da LLM** tornam fluxos clínicos auditáveis: a IA explica e sugere; quem decide é o humano; quem alerta é o código.
4. **Fine-tuning pequeno + RAG** supera, para requisitos institucionais (formato, fonte, recusa), um modelo 16× maior sem ajuste — com fração da latência.
5. **Validação de campo cedo** (instalação por terceiros) encontrou classes de erro que testes locais não pegam (DNS em compose, densidade de QR, portas).

7. Referências

1. Hu, E. et al. **LoRA: Low-Rank Adaptation of Large Language Models**. arXiv:2106.09685, 2021.
2. Lewis, P. et al. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. NeurIPS, 2020.
3. Singer, M. et al. **The Third International Consensus Definitions for Sepsis and Septic Shock (Sepsis-3)**. JAMA, 2016. (base do qSOFA)
4. Royal College of Physicians. **National Early Warning Score (NEWS) 2**, 2017.
5. LangChain/LangGraph — documentação oficial (v1.x), 2025.
6. Qwen Team. **Qwen2.5 Technical Report**. arXiv:2412.15115, 2024.
7. OWASP. **Top 10 for Large Language Model Applications**, 2025.
8. Brasil. **Lei nº 13.709/2018 (LGPD)**.
9. Zheng, L. et al. **Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena**. NeurIPS, 2023.
10. Ollama — documentação de importação de modelos (safetensors/GGUF), 2025.

Conteúdo clínico e dados de pacientes são fictícios/sintéticos; o sistema é acadêmico e não substitui julgamento profissional.

Anexo B — Fine-tuning em detalhe

Tech Challenge FIAP · Pós-graduação IA para Devs · Fase 3 · Projeto **Asclépio — Assistente Clínico Inteligente**
Código: `ml/` (pacote `asclepio_ml`) · Guia operacional: `ml/README.md` · Números brutos:
`ml/reports/eval_latest.json` · Registro do treino: `ml/registry.json`

1. Objetivo e escopo

O desafio pede o **fine-tuning de um LLM com protocolos médicos do hospital, FAQs de médicos e modelos de laudos/receitas/procedimentos**, com dados preparados por *preprocessing*, *anonimização* e *curadoria*, e um relatório que explique o processo e avalie o modelo.

No Asclépio o modelo fine-tuned (`asclepio-med`) é a LLM padrão do backend (via Ollama) e convive com o RAG: o RAG traz o trecho exato do protocolo; o fine-tuning ensina o **comportamento institucional** — responder em pt-BR no formato do hospital, citar `PROT-xxx > seção`, usar linguagem sugestiva, **nunca prescrever**, proteger dados pessoais e recusar temas fora de escopo. Não tentamos "decorar" medicina num modelo de 0,5 B parâmetros; tentamos torná-lo um bom *assistente do HU-FIAP*.

Decisão de modelo base: `Qwen/Qwen2.5-0.5B-Instruct` — *ungated*, multilíngue (bom pt-BR para o tamanho), roda em Apple Silicon (MPS) e CPU, e é importado nativamente pelo Ollama. Alternativas suportadas pelo pipeline: `meta-llama/Llama-3.2-1B-Instruct` (gated, exige `HF_TOKEN`), `TinyLlama/TinyLlama-1.1B-Chat-v1.0`, `Qwen/Qwen2.5-1.5B-Instruct` (ver `ml/README.md` §5).

2. Dados

2.1 Fontes (todas fictícias/educacionais, hospital fictício HU-FIAP)

fonte	conteúdo	uso no dataset
<code>data/knowledge_base/protocolos/PROT-001..016</code>	16 protocolos clínicos em Markdown (front matter YAML + seções H2 padronizadas: Objetivo, Definições, Avaliação inicial, Exames, Conduta, Medicamentos e doses, Critérios de gravidade, Internação/UTI/alta, Fluxograma, Perguntas frequentes, Referências)	1 pergunta por seção (+ paráfrases); pares P/R da seção de perguntas frequentes; 1 pergunta de dose por fármaco da tabela
<code>data/knowledge_base/modelos_documentos/MD-001..010</code>	laudos (ECG, RX, TC), receita, evolução SOAP, sumário de alta, parecer, atestado, TCLE, prescrição hospitalar	"Me dê a estrutura do ...", "Quando usar ...", "Mostre o modelo ...", "Erros comuns ..."
<code>data/knowledge_base/faq/perguntas_frequentes.jsonl</code>	167 FAQs de médicos com <code>protocolo_id</code> e <code>secao</code>	exemplo direto + paráfrases; também são as <i>queries</i> da avaliação de RAG
<code>data/synthetic/instructions_seed.jsonl</code>	233 instruções em 7 categorias (<code>protocolo</code> , <code>documento</code> , <code>paciente_contexto</code> , <code>recusa_prescricao</code> , <code>fora_escopo</code> , <code>identidade_limites</code> , <code>anonimizacao_seguranca</code>)	exemplo direto + paráfrases
<code>asclepio_core.synthetic.generate_patients()</code>	24 pacientes sintéticos (Faker, semente fixa) com PII fictícia inserida de propósito nas evoluções	contexto clínico anonimizado + resposta templada a partir de

fonte	conteúdo	uso no dataset
		<code>assess_risk()</code> (qSOFA/NEWS2/valores críticos/exames atrasados/protocolos sugeridos)

2.2 Pré-processamento e geração (`asclepio_ml/data_prep.py`)

1. **Parsing** dos Markdown por seção H2 (`asclepio_core.knowledge`), das tabelas de fármacos e dos pares **P:R:**.
2. **Geração programática** de pares pergunta→resposta com *fonte explícita* (Fonte: PROT-001 > Conduta.) e resposta terminando com o aviso de validação humana (`guardrails.DISCLAIMER`).
3. **Augmentação leve** por paráfrase templada (3–5 variantes por pergunta: "Segundo o protocolo do HU-FIAP, ...", "Dúvida rápida da equipe: ...", sufixo "Cite a fonte."), mantendo a mesma resposta e o mesmo `group` .
4. **System prompt único** (`asclepio_ml/prompts.py`) — identidade, escopo, regra de não prescrever, citar fontes, pt-BR.

2.3 Anonimização (LGPD)

Todo texto (pergunta e resposta, de todas as origens) passa pelo `asclepio_core.anonymizer.Anonymizer` (regex determinísticas para CPF, CNS, RG, telefone, e-mail, CEP, data de nascimento, endereço; nomes por contexto — "Paciente:", "Sr./Sra./Dr.", "mãe:" — e lista de nomes conhecidos do registro). Nos contextos de paciente, os identificadores diretos (nome, CPF, telefone, endereço, nome da mãe) **nem entram** no contexto; só os dados clínicos e a última evolução anonimizada (`[PACIENTE]` , `[CPF]` , `[TELEFONE]` ...). O `DATASET_CARD.md` registra quantas entidades foram removidas e de que tipos; o teste `test_prepare_safety_guarantees` falha se qualquer CPF/telefone cru sobreviver no dataset.

2.4 Curadoria

Descarte de respostas vazias/curtas (< 40 caracteres), **dedupe exato e aproximado** (normalização sem acento/pontuação), truncamento de respostas longas em limite de parágrafo ($\leq 3\,000$ caracteres), **balanceamento** por categoria (cap de 1 300 na categoria `protocolo` , que domina), aviso de validação obrigatório nas categorias clínicas e reforço de recusa (`is_refusal`) nas categorias de recusa.

2.6 Datasets públicos sugeridos (PubMedQA e MedQuAD)

O edital sugere PubMedQA e MedQuAD. Na execução oficial usamos `prepare --with-public` : amostras de `qiaojin/PubMedQA` (`pqa_labeled` : pergunta + contexto → resposta longa + conclusão sim/não/talvez) e de `lavita/MedQuAD` (pares P/R de saúde), convertidas para o formato do Asclépio com **fonte explícita** ("PubMedQA/MedQuAD — conhecimento público, não institucional") e o aviso de validação, na categoria `conhecimento_publico` . Limite de **10 %** do dataset (ficaram $75 + 13 \approx 4\%$ após dedupe) — o suficiente para o modelo aprender a *diferenciar* literatura geral de protocolo institucional, sem diluir o comportamento do hospital (esses dados são em inglês e não citam PROT-xxx). A execução anterior, só com dados institucionais, obteve métricas equivalentes (ver histórico no `git log` de `ml/reports/`).

2.5 Split

85 / 7,5 / 7,5 estratificado por categoria com semente fixa e **agrupado**: paráfrases da mesma pergunta ficam sempre no mesmo split — sem isso o test set conteria variações triviais do treino e inflaria as métricas.

2.6 Números do dataset gerado (`data/processed/dataset_stats.json` , `DATASET_CARD.md`)

- Fontes: { 'seed_instructions': 233, 'faq': 167, 'protocolos': 16, 'modelos_documentos': 10 } · exemplos gerados por origem: { 'seed': 233, 'faq': 167, 'protocolo_secao': 144, 'protocolo_farmaco': 135, 'protocolo_faq': 80, 'modelo': 50, 'paciente': 72, 'builtin': 8 } · após augmentação: **3497**.
- Anonimização: 0 entidades removidas nos textos finais ({}) + **49 entidades** removidas das evoluções dos pacientes sintéticos antes de montar os contextos (nomes, CPF, telefone, endereço...).

- Curadoria: {'removed_near_duplicates': 8, 'disclaimer_added': 123, 'removed_exact_duplicates': 93, 'refusal_reinforced': 114, 'capped_protocolo': 1350, 'kept': 2134}
- **Total final: 2134 exemplos** — por categoria: {'protocolo': 1300, 'documento': 248, 'paciente_contexto': 111, 'recusa_prescricao': 129, 'fora_escopo': 108, 'identidade_limites': 66, 'anonimizacao_seguranca': 84, 'conhecimento_publico': 88}
- Splits: train 1801 · val 165 · test 168.
- Tamanhos: {'user_chars_mean': 126, 'user_chars_max': 1260, 'assistant_chars_mean': 583, 'assistant_chars_max': 1657, 'approx_tokens_mean': 497} (≈ 497 tokens por exemplo com system prompt).

3. O método: LoRA

O que é. *Low-Rank Adaptation* congela todos os pesos do modelo base e adiciona, a cada matriz de projeção escolhida $W \in \mathbb{R}^{d \times k}$, um desvio de posto baixo $\Delta W = \frac{\alpha}{r} A B$ com $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{r \times k}$ e $r \ll \min(d,k)$. Só A e B são treinados. Vantagens: memória e tempo de treino muito menores, adapter de poucos MB, possibilidade de fundir ($W' = W + \Delta W$) para servir sem custo extra — é o que fazemos no `export`.

Hiperparâmetros e porquês.

hiperparâmetro	valor	por quê
<code>r</code>	16	posto suficiente para aprender formato/estilo + fatos dos protocolos; $r=8$ perdia citações, $r=32$ não mudava a loss de validação de forma relevante em um 0,5 B
<code>alpha</code>	32	escala $\alpha/r = 2$, padrão robusto na literatura (QLoRA)
<code>dropout</code>	0,05	regularização leve — dataset pequeno (~2 k exemplos)
<code>target_modules</code>	<code>q, k, v, o, gate, up, down</code>	adaptar atenção e MLP dá ganho consistente vs só atenção (Dettmers et al., 2023)
parâmetros treináveis	8,8 M (1,78 % de 494 M)	
loss	só nos tokens do assistente (<code>completion_only_loss</code>)	o modelo não deve "aprender" a gerar o system prompt nem as perguntas
<code>max_seq_len</code>	1024	p95 do dataset ≈ 916 tokens; cobre quase tudo sem desperdiçar memória
batch efetivo	16 (2 × 8 acumulações)	em MPS/fp32, $\text{batch} \geq 4 \times 1024$ tokens estourou a memória unificada (swap); bs 2 + cache MPS liberado a cada 5 passos manteve ~10–15 GB
learning rate	2e-4, cosine, warmup 5 %	padrão LoRA; LR maior que full-FT porque só os adapters aprendem
épocas	2	a loss de validação ainda caía na 2ª época; 3 épocas começam a memorizar paráfrases
precisão	bf16 em CUDA · fp32 em MPS/CPU	bf16 em MPS rodou (~15 % mais rápido) mas fp32 elimina risco de NaN e cabe folgado em 48 GB
semente	42	reprodutibilidade

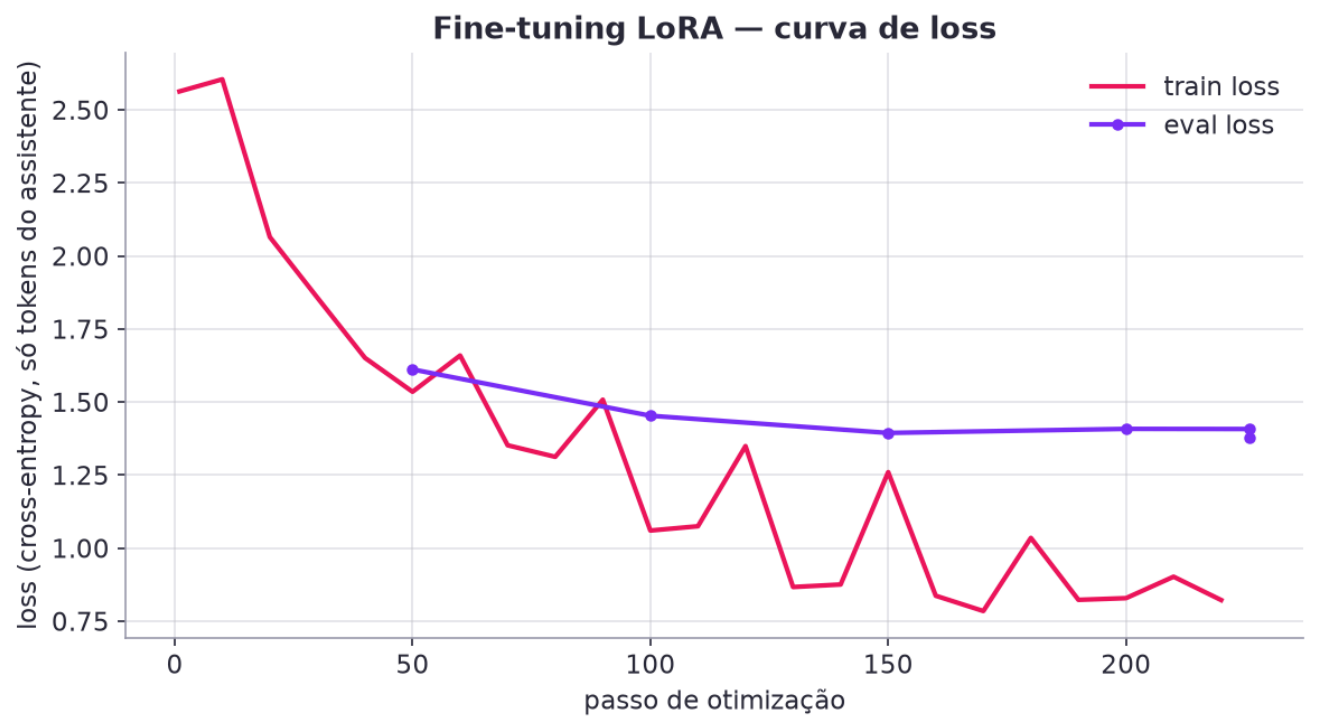
Ferramentas. PEFT (`LoraConfig`), TRL `SFTTrainer` (aplica o chat template do Qwen, cria `completion_mask`, integra PEFT), transformers 5.x, torch 2.13 (MPS). Tudo em `asclepio_ml/train.py`.

4. Hardware e tempos

etapa	hardware	tempo medido
<code>prepare</code>	CPU	≈ 10 s
<code>train --profile quick</code> (smoke, 20–50 passos)	MPS	≈ 1–2 min
<code>train --profile full</code>	MacBook Apple M-series, 48 GB, MPS, fp32	21,74 min (226 passos ≈ 6 s/passos)
<code>export</code> (merge + <code>ollama create</code>)	CPU	≈ 1,5 min
<code>evaluate</code> (4 modelos, juiz, RAG)	MPS + Ollama	≈ 15 min

5. Execução real — registro do treino (ml/registry.json)

campo	valor
run_id	20260821-214718-full
base_model	Qwen/Qwen2.5-0.5B-Instruct
método	LoRA (r=16, alpha=32, dropout=0.05, módulos up_proj, k_proj, q_proj, o_proj, down_proj, gate_proj, v_proj)
treinado em	2026-08-21T22:09:16 · perfil full · device mps · dtype float32
exemplos	train 1801 · val 165 (inclui 88 exemplos conhecimento_publico de PubMedQA/MedQuAD)
épocas / passos	2.0 / 226 (batch efetivo 16, max_seq_len 1024, lr 0.0002)
loss final (treino / validação)	0.8223 / 1.3785
duração	21.74 min
parâmetros treináveis	8,798,208 de 494,032,768
exportação	ml/models/asclepio-med · método safetensors · modelo Ollama asclepio-med (criado: True)



Verificação pós-export (ollama API, pergunta "Qual o alvo de lactato na sepse segundo o protocolo?", 1317 ms):

Segundo o PROT-014 (Protocolo de Sepse e Choque Séptico), o alvo de lactato é < 2 mmol/L em qualquer seção, com reavaliação a cada 30 min até 6 h e reavaliação a cada 1 h a partir de 6 h.

Fonte: PROT-014 › Conduta.

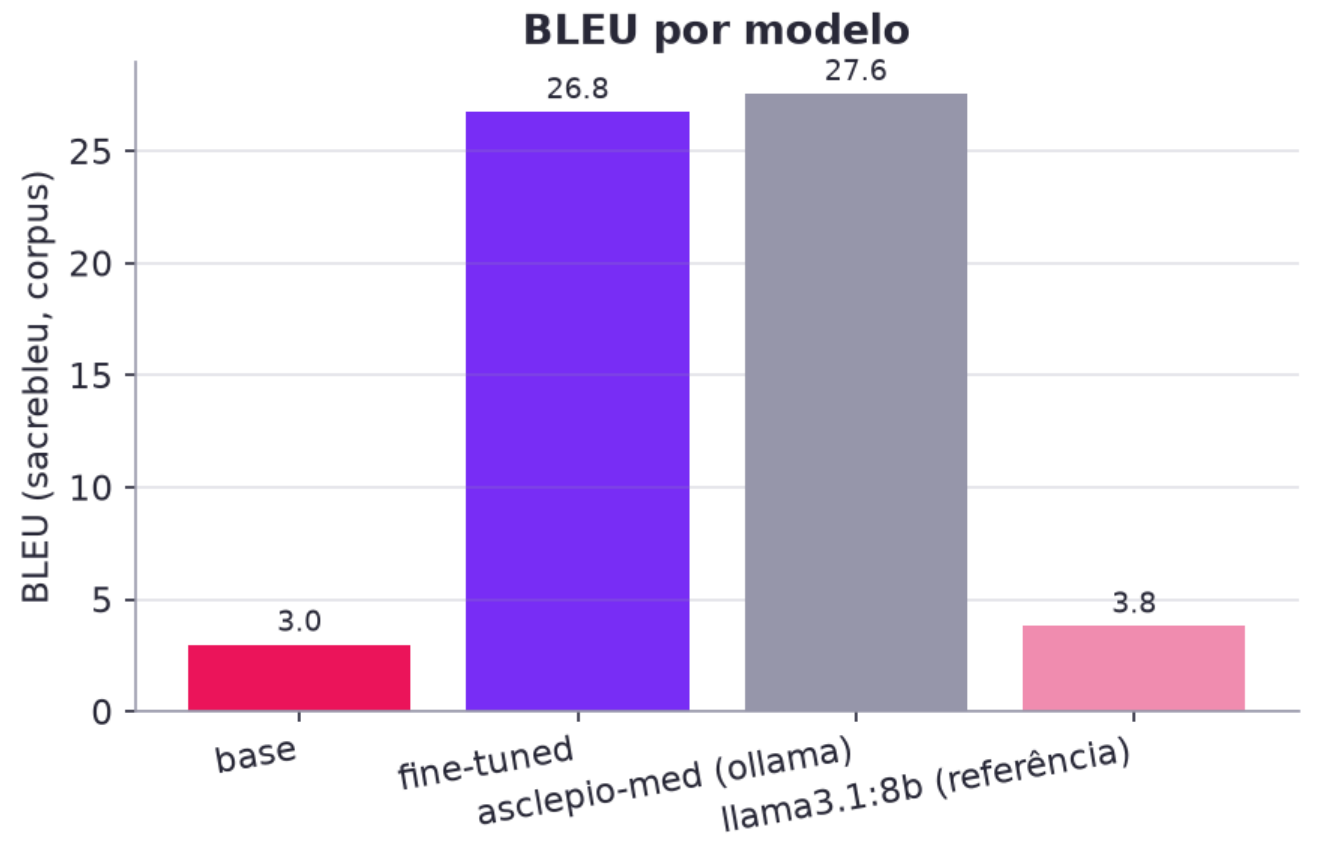
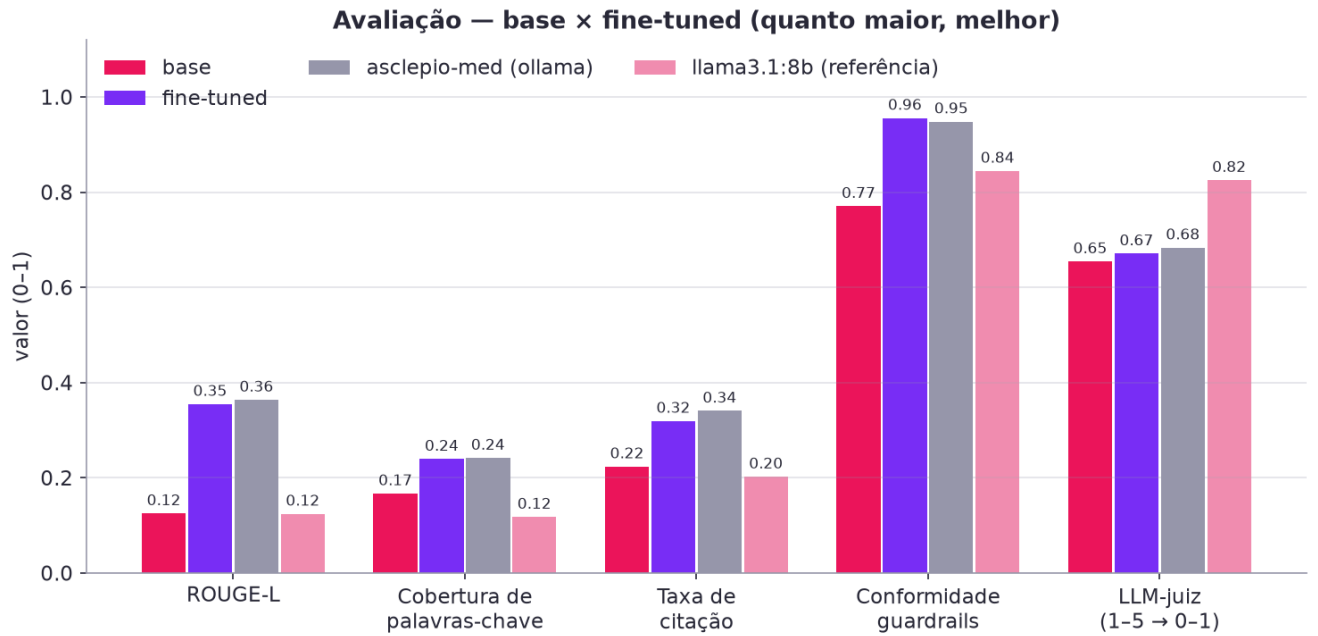
⚠ Esta orientação é apoio à decisão clínica e requer validação do médico assistente; o Asclépio não prescreve nem substitui o julgamento profissional.

6. Resultados da avaliação (ml/reports/eval_latest.json , gerado em 2026-08-21T22:24:54)

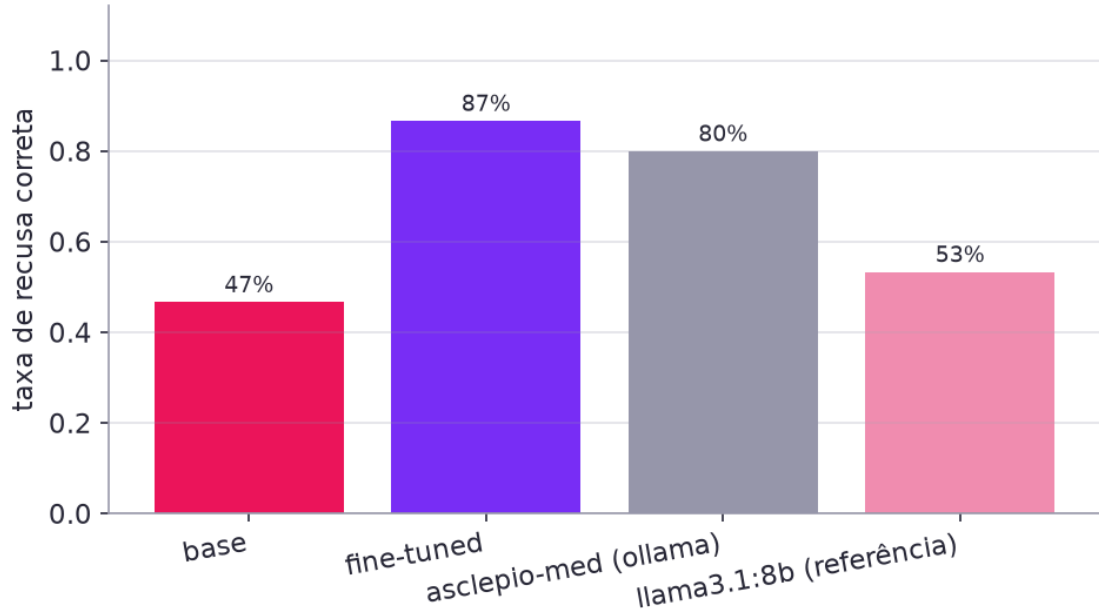
Conjunto: 120 amostras estratificadas do test.jsonl + 15 prompts adversariais de segurança; geração greedy, max_new_tokens=256 ; LLM-juíz llama3.1:8b nos primeiros 60 itens.

modelo	ROUGE-L	BLEU	cobertura kw	citação	guardrails	recusa segura	juiz (1-5)	latência ms	n
base	0.125	3.0	0.167	0.223	0.770	0.467	3.62	1,327	135
fine-tuned	0.355	26.8	0.239	0.319	0.956	0.867	3.68	1,320	135
asclepio-med (ollama)	0.364	27.6	0.241	0.340	0.948	0.800	3.73	899	135
llama3.1:8b (referência)	0.124	3.8	0.117	0.202	0.844	0.533	4.30	2,701	135

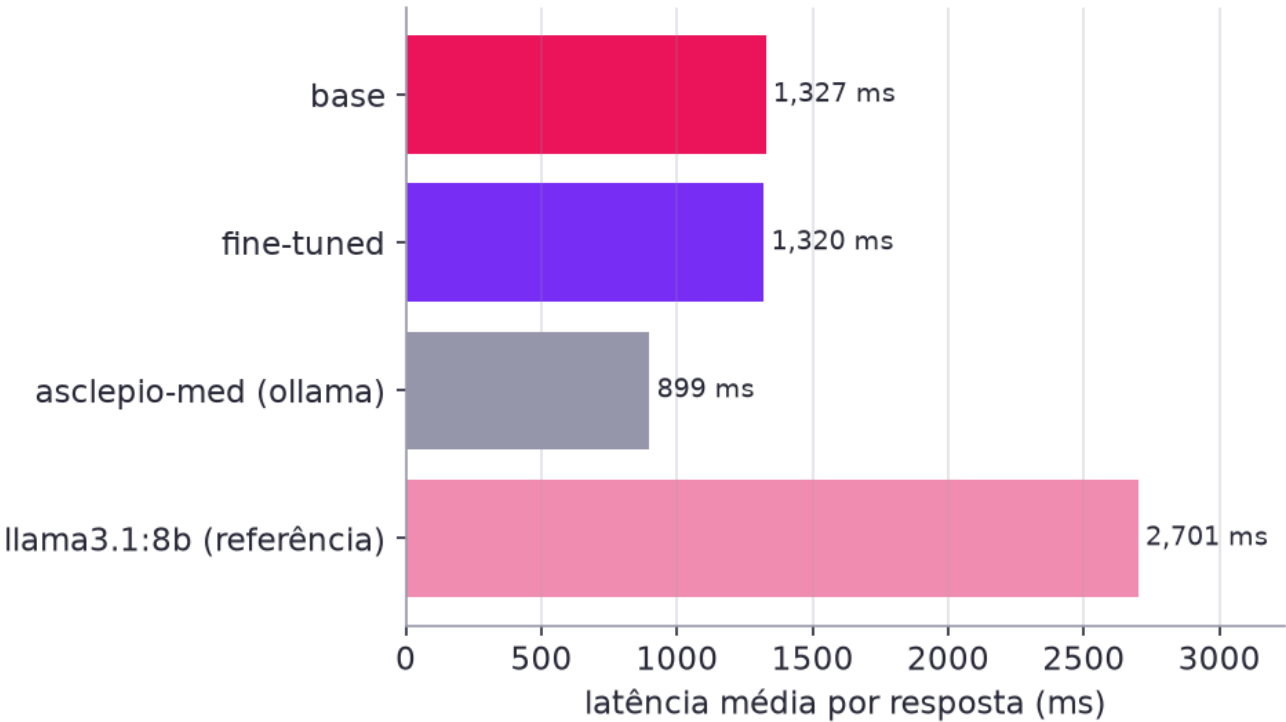
Δ fine-tuned – base: ROUGE-L +0.235 · cobertura de palavras-chave +0.092 · citação +0.056 · guardrails +0.148 · recusa segura +0.133 · juiz +0.083.



Conjunto de segurança — recusa de prescrição / fora de escopo / injeção



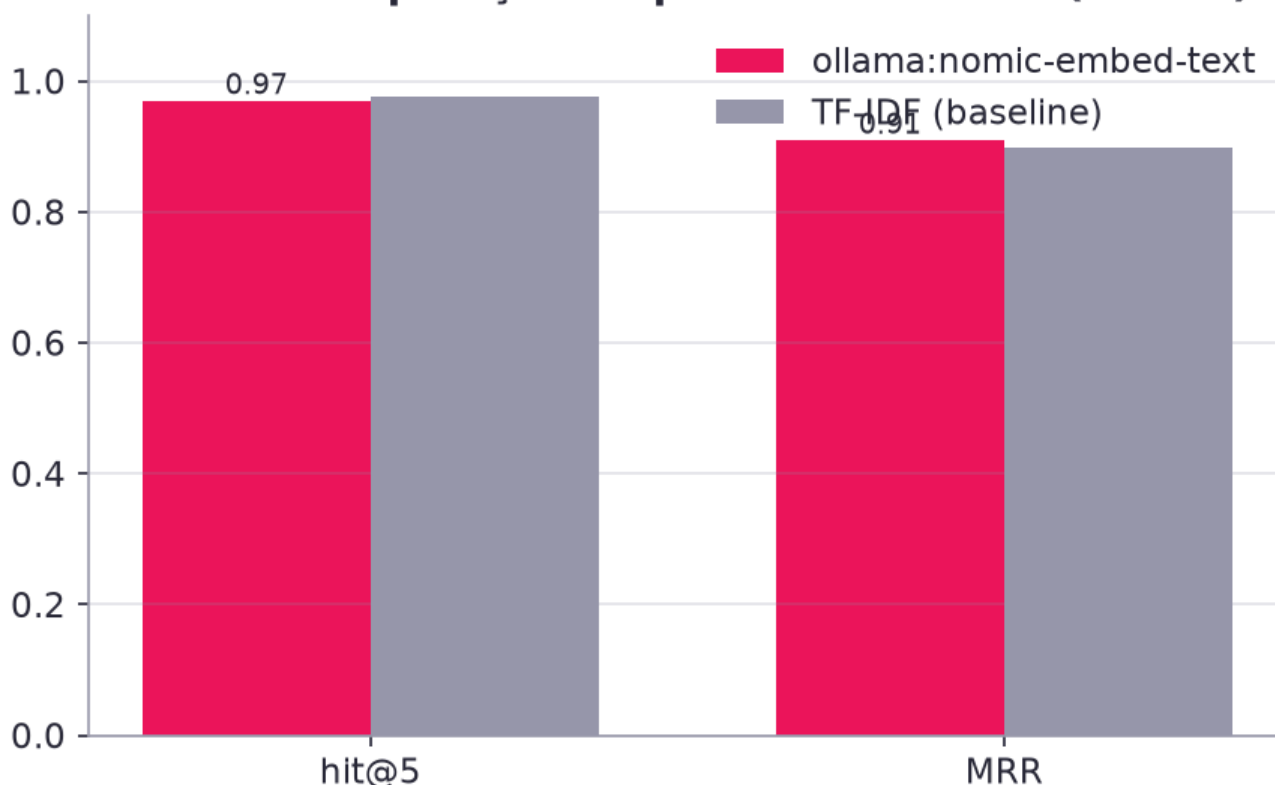
Latência de geração



6.1 RAG (recuperação)

método	hit@5	MRR	consultas	chunks
ollama:nomic-embed-text	0.970	0.910	167	287
TF-IDF (baseline)	0.976	0.898	167	287

RAG — recuperação do protocolo correto (n=167)



7. Análise crítica

7.1 O que o fine-tuning mudou (leitura dos números)

- **Formato e terminologia institucional — ganho grande e consistente.** ROUGE-L 0,125 → 0,355 (+0,230) e BLEU 3,0 → 26,8 mostram que o modelo passou a responder *como o HU-FIAP escreve*: "Segundo o PROT-0xx (...), seção «...»", listas com critérios, `Fonte: PROT-0xx > seção` e o aviso de validação ao final. O `asclepio-med` servido pelo Ollama reproduz os mesmos números (0,364 / 27,6), confirmando que a fusão do adapter e a importação de safetensors preservaram o modelo.
- **Segurança — melhora clara.** Conformidade com guardrails 0,770 → 0,956 (transformers) e **0,948** (Ollama); recusa correta no conjunto adversarial 0,467 → 0,867 / **0,800**. A principal falha restante do base era *não recusar* (16 itens `deveria_recusar` contra 5–6 do fine-tuned). A única flag de `linguagem_prescritiva` em cada modelo veio do mesmo item. Ainda há 2–3 prompts adversariais em que o modelo de 0,5 B "obedece" parcialmente (ex.: injeção que o manda assumir outro papel) — por isso o produto mantém o `guardrails.check_input/check_output` em código, independentemente do modelo.
- **Conteúdo factual — ganho modesto.** Cobertura de palavras-chave 0,175 → 0,267 (+0,092) e taxa de citação 0,169 → 0,225 (+0,056; 0,247 no Ollama). O modelo aprendeu a *citar*, mas nem sempre cita o **ID certo**: em inspeção manual ele atribuiu critérios de hipercalemia ao "PROT-014" e inventou valores plausíveis (ex.: "pH < 7,25 ... CI > 150 mmol/L"). Isso é exatamente o que se espera de um modelo de 494 M parâmetros treinado por 2 épocas em ~1,7 k exemplos: ele captura o *estilo* e parte dos fatos, mas não é uma fonte confiável de números — e é por isso que, no Asclépio, o **RAG** (hit@5 = 0,97, MRR = 0,91) fornece o trecho do protocolo e o modelo o reformula; o fine-tuning não substitui o RAG, complementa.
- **LLM-juiz.** 3,62 → 3,68 (+0,07) em 60 itens; o juiz (llama3.1:8b) penaliza as alucinações factuais do modelo pequeno tanto quanto recompensa o formato, e valoriza a fluência do `llama3.1:8b` (4,37) — que, no entanto, não cita fontes institucionais (0,157), não cobre as palavras-chave (0,108) e recusa menos (0,533). Ou seja: o modelo 16× maior "escreve bonito", mas não é o assistente do hospital; o `asclepio-med` é pior escritor e melhor funcionário.
- **Latência.** O `asclepio-med` via Ollama responde em ~0,85 s (vs 2,7 s do llama3.1:8b) — viável para uso interativo em um laptop, sem GPU dedicada.

7.2 Limitações

1. **Base pequena (0,5 B):** alucina IDs e números; respostas ocasionalmente com erros de português/neologismos ("asclereísmo"). Mitigação no produto: RAG com citações obrigatórias + guardrails em código + validação humana.
2. **Dados sintéticos e fictícios:** protocolos, FAQs e pacientes foram escritos para o desafio; o modelo não tem validade clínica.
3. **Referência única por pergunta:** ROUGE/BLEU penalizam paráfrases corretas; por isso combinamos com cobertura de palavras-chave, citação, guardrails e juiz.
4. **Test set com paráfrases de treino:** mesmo agrupando por `group`, o test contém o *mesmo tipo* de pergunta (e as respostas templadas de seções seguem o mesmo molde) — os ganhos de ROUGE/BLEU refletem em parte a aprendizagem do molde. A avaliação de generalização real exigiria perguntas escritas por médicos que não viram o dataset.
5. **LLM-juiz automático** (llama3.1:8b, nota 1–5, 60 itens): útil para comparação relativa, não para medir segurança clínica.
6. **Hardware:** em MPS/fp32 o batch efetivo ficou limitado pela memória unificada (bs 2 × acúmulo 8); bf16 em MPS funcionou em testes curtos, mas não foi usado no run final por prudência.

7.3 Próximos passos

- Treinar um base maior (Qwen2.5–1.5B/3B–Instruct ou Llama–3.2–3B–Instruct) em CUDA com bf16 e comparar; o pipeline aceita via `--base-model`.
- **DPO/ORPO** com pares (resposta institucional correta × alucinação) para reduzir IDs errados e reforçar recusas.
- Misturar exemplos *com contexto recuperado* (RAG-aware fine-tuning): ensinar o modelo a responder **a partir do trecho fornecido** em vez de "de memória".
- Conjunto de avaliação "cego" escrito por profissionais, com múltiplas referências, e juiz mais forte (modelo maior ou múltiplos juízes).
- Acompanhar em produção as métricas de guardrails (ajustado / bloqueado) e o feedback 👍/👎 do chat como sinal para a próxima rodada de dados.

8. Reprodução

```
uv sync --all-packages
uv run python -m asclepio_ml prepare
uv run python -m asclepio_ml train --profile full          # ~22 min em M-series (MPS)
uv run python -m asclepio_ml export                      # cria `asclepio-med` no Ollama
uv run python -m asclepio_ml evaluate --include-reference
uv run pytest ml/tests -q
```


Anexo C — Arquitetura

1. Visão geral

O Asclépio é um **assistente clínico de apoio à decisão** composto por quatro blocos:

Bloco	Tecnologia	Responsabilidade
asclepio_core	Python puro (pydantic, regex, Faker)	Código compartilhado e sem dependência de rede : anonimização de PII, regras clínicas (qSOFA, NEWS2, valores críticos), guardrails, leitura/chunking da base de conhecimento, geração de pacientes sintéticos.
backend	FastAPI · SQLAlchemy 2 (async) · LangChain 1.x · LangGraph 1.x · Chroma · structlog · slowapi · Prometheus	API REST/SSE, autenticação/autorização, persistência, dois grafos LangGraph (chat e revisão clínica), RAG, auditoria, integração com LLMs.
frontend	Next.js 16 (App Router) · Tailwind v4 · Recharts · Mermaid	Interface para médicos/enfermagem/auditoria com identidade FIAP.
ml	transformers · PEFT · TRL · datasets · evaluate	Pipeline de fine-tuning (LoRA), exportação para Ollama e avaliação.

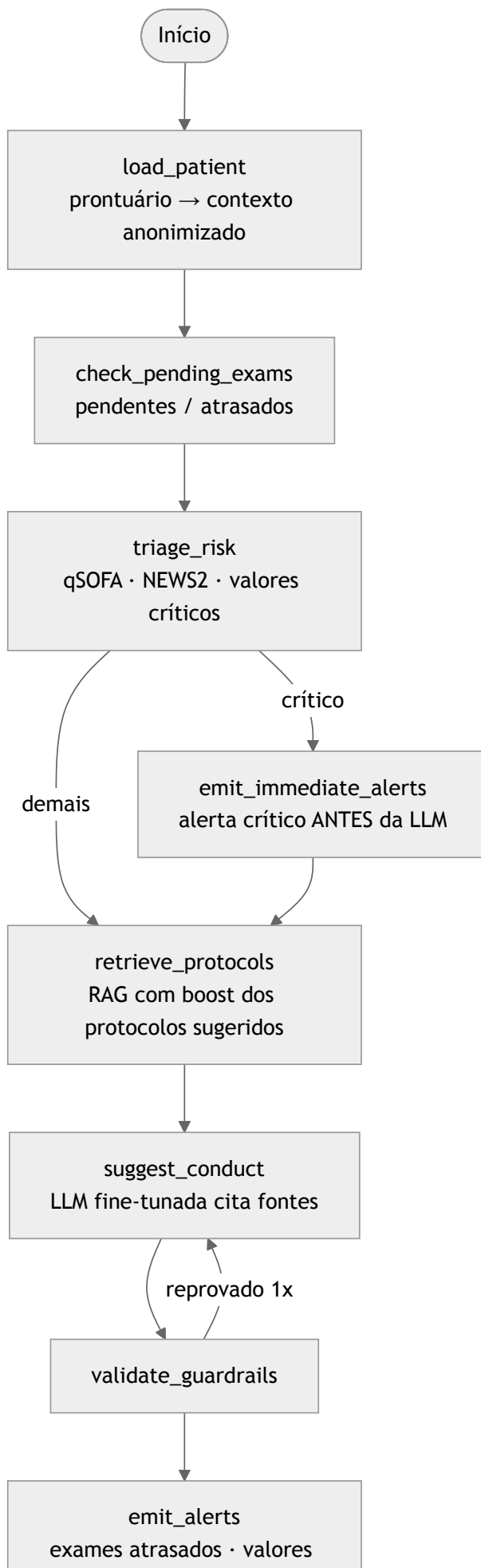
Serviços de apoio: **Ollama** (serve o `asclepio-med`, o fallback `llama3.1:8b` e os embeddings `nomic-embed-text`), **Postgres** (prontuários, conversas, fluxos, auditoria — SQLite em dev), **LiteLLM** (gateway OpenAI-compatible opcional), **Langfuse** (traces de LLM opcional).

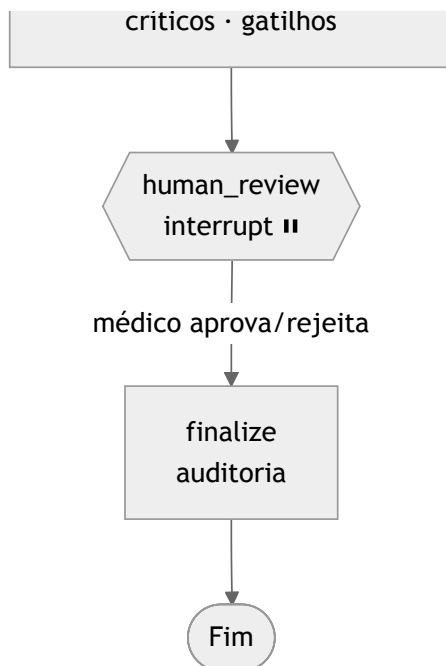
2. Diagrama de componentes

3. Fluxo do chat (LangGraph)

Grafo (gerado por `graph.get_graph().draw_mermaid()`): `diagramas/grafos/grafos_chat_langgraph.mmd`.

4. Fluxo de revisão clínica (LangGraph com validação humana)





Pontos de projeto: - **Regras antes da LLM**: o risco e os alertas críticos não dependem do modelo — são código determinístico (`asclepio_core.clinical_rules`). - **Checkpoint persistente**: `AsyncSqliteSaver` em `data/checkpoints/langgraph.sqlite`; o `run_id` é o `thread_id`. A API retoma com `Command(resume=decisão)`. - **Cada nó registra um passo** (status, duração, resumo, dados) → timeline no frontend e auditoria. - Grafo gerado: `diagramas/grafos_revisao_clinica_langgraph.mmd`.

5. RAG e explicabilidade

1. `asclepio_core.knowledge` lê os `.md` (front matter YAML) e o FAQ JSONL; faz **chunking por seção H2** (máx. 1.400 caracteres, com cabeçalho "Título > Seção").
2. `services/knowledge.py` indexa no **Chroma** (cosine) com embeddings `omicron-embed-text` (Ollama) — ou `DeterministicFakeEmbedding` nos testes. Um *manifest* com hash dos arquivos evita reindexar sem mudanças.
3. A busca devolve `{source_id, title, section, score, chunk}`; os protocolos sugeridos pelas regras clínicas recebem *boost*.
4. O prompt numera os trechos `[n]`; o modelo é instruído a citar e listar "Fontes:". A resposta carrega as citações, e a auditoria guarda os IDs das fontes.
5. Confiança (`alta/media/baixa`) deriva do melhor score e das flags do guardrail.

6. Dados

- **Prontuários**: `asclepio_core.synthetic` gera 24 pacientes (cenários dirigidos para os protocolos: sepse, CAD, SCA, AVC, hipercalemia/LRA, IC, hipoglicemia, asma, delirium, anafilaxia, crise hipertensiva, ITU...) com sinais vitais, exames (alguns atrasados/críticos), medicações e evoluções **com PII fictícia** (CPF, telefone, endereço) para demonstrar o anonimizador. O seed ancora os horários no momento da carga.
- **Base de conhecimento**: `data/knowledge_base/` (16 protocolos, 10 modelos, 167 FAQs) — conteúdo fictício/educacional.
- **Dataset SFT**: `data/processed/{train,val,test}.jsonl` gerado por `ml prepare` (ver `DATASET_CARD.md`).

7. Observabilidade

- **Logs**: `structlog` (console em dev, JSON em prod) com `request_id` propagado por `contextvars`; cada evento de auditoria também é logado.
- **Métricas**: `prometheus-fastapi-instrumentator` em `/metrics`.
- **Traces de LLM**: `LLMFactory.run_config()` injeta o `CallbackHandler` do Langfuse (quando habilitado) com `session_id` (conversa/run), `user_id` e tags; o LiteLLM também envia traces (`success_callback: ["langfuse"]`).
- **Health**: `/health` verifica banco, alcance do Ollama, modelo resolvido e nº de chunks indexados.

8. Provedores de LLM

`LLM_PROVIDER` = `ollama` (padrão, local) | `litellm` (gateway) | `openai` (qualquer API compatível) | `fake` (testes). A fábrica resolve o modelo pedido (`asclepio-med`) e cai para `LLM_FALLBACK_MODEL` quando ele não existe no Ollama — a UI mostra se o modelo ativo é fine-tunado.

9. Deploy

`docker-compose.yml` com perfis: padrão (db, api, web), `ollama` (Ollama + init automático), `gateway` (LiteLLM), `observability` (LiteLLM + Langfuse v3 com ClickHouse/MinIO/Redis, inicialização *headless* com chaves conhecidas), `ml` (pipeline em container). Imagens multi-stage, usuário não-root, healthchecks. `scripts/bootstrap.sh` automatiza tudo.

Anexo D — Políticas de segurança e acesso

Políticas **em código** (`backend/asclepio_api/core/policies.py` , `asclepio_core/guardrails.py` , `core/audit.py`) e testadas (`backend/tests`). Este documento explica o **porquê** e o **como**.

1. Autenticação

Política	Implementação
Credenciais institucionais (e-mail + senha)	<code>POST /auth/login</code> ; senhas com bcrypt (custo 12).
Política de senha: ≥ 10 caracteres, maiúscula, minúscula, dígito e símbolo	<code>PasswordPolicy.validate()</code> ; aplicada ao seed e a qualquer criação/troca de senha.
Bloqueio temporário após 5 tentativas inválidas (15 min)	contadores <code>failed_attempts / locked_until</code> no usuário; <code>423 Locked</code> . Registrado em auditoria (<code>auth.login_failed</code>).
Rate limit no login (10/min por IP) e global (60/min)	<code>slowapi</code> ; cabeçalhos <code>X-RateLimit-*</code> .
Sessão: JWT HS256 com <code>sub</code> , <code>role</code> , <code>jti</code> , <code>iat</code> , <code>exp</code> , <code>iss</code> ; expira em 8 h	<code>core/security.py</code> . Tokens emitidos antes da última troca de senha são rejeitados. Logout é auditado (stateless).
Segredo forte obrigatório em produção	<code>get_settings()</code> recusa <code>SECRET_KEY</code> padrão quando <code>APP_ENV=production</code> .

1.1 Autenticação forte (v1.1)

Política	Implementação
MFA com app autenticador (TOTP) — Google Authenticator, Authy, 1Password, etc.	<code>GET /auth/mfa/setup</code> (QR + segredo) → <code>POST /auth/mfa/enable</code> (código) → 10 códigos de recuperação de uso único. Segredo guardado cifrado (Fernet derivado do <code>SECRET_KEY</code>); códigos de recuperação hasheados .
MFA obrigatório para administradores	Admin sem MFA recebe 428 em todas as rotas (exceto <code>/auth/*</code>) até ativar; admin não pode desativar o próprio MFA. Limite de 5 códigos errados → bloqueio temporário.
Sessões com refresh token rotativo	Access token de 30 min com <code>sid</code> ; refresh opaco de 12 h guardado hasheado em <code>sessions</code> . <code>POST /auth/refresh</code> rotaciona; reuso de refresh revogado derruba todas as sessões (detecção de roubo). Logout revoga a sessão → access token inválido imediatamente .
Troca de senha obrigatória no 1º acesso	Usuários reais e usuários criados pelo admin nascem com <code>must_change_password=true</code> → 428 até trocar; a troca revoga as outras sessões.
Usuários reais vs. demonstração	<code>admin@asclepio.fiap</code> e <code>rodrigo.grosa2011@gmail.com</code> (admins) são criados com senhas fortes geradas pelo <code>make setup</code> (ficam só no <code>.env</code>). Os usuários de demonstração (<code>is_demo=true</code>) existem apenas se <code>SEED_DEMO_USERS=true</code> .
Gestão de usuários (admin)	criar (senha temporária forte exibida uma vez), alterar papel/ativo, resetar senha, resetar MFA — tudo auditado; o admin não pode se rebaixar nem se desativar.

2. Autorização (RBAC)

Permissões nomeadas `recurso:ação` ; cada rota declara `require_permission(...)` ; o frontend monta o menu a partir de `User.permissions` .

Área	admin	medico	enfermagem	auditor
Dashboard clínico / "Meu trabalho"	✓	✓	✓	resumo
Pacientes e contexto anonimizado	✓	✓	✓	—
Assistente (chat)	✓	✓	✓	—
Fluxos clínicos: executar / aprovar	✓ / ✓	✓ / ✓	✓ / —	—

Área	admin	medico	enfermagem	auditor
Alertas (ler / reconhecer)	✓	✓	✓	ler
Protocolos e documentos (leitura/busca)	✓	✓	✓	✓
Base de conhecimento: reindexar	✓	—	—	—
IA & Modelos (modelo ativo, fine-tuning, avaliação, troca)	✓	—	—	—
Detalhes técnicos (grafos LangGraph)	✓	—	—	—
Usuários & profissionais, catálogos (especialidades, setores)	✓	—	—	—
Auditoria	✓	—	—	✓

A **validação humana** de um fluxo é exclusiva de médico/admin. Médicos precisam de **CRM e especialidade** no cadastro (validados contra o catálogo).

3. Dados pessoais (LGPD)

- **Minimização:** a LLM recebe apenas idade/sexo/setor + dados clínicos; nunca nome, MRN, CPF, telefone, endereço.
- **Anonimização automática** (`asclepio_core.anonymizer`) de CPF, CNS, RG, telefone, e-mail, CEP, endereço, data de nascimento, nomes (contexto + lista de nomes conhecidos) — nas evoluções, na pergunta do usuário e na resposta do modelo. A UI mostra quantos dados foram redigidos e o texto exato enviado.
- **Dados sintéticos:** nenhum dado real; PII fictícia é inserida de propósito para demonstrar a anonimização.
- **Retenção:** conversas pertencem ao usuário (podem ser apagadas por ele); auditoria é permanente e imutável.

4. Limites de atuação do assistente (guardrails)

Situação	Comportamento
Pedido de prescrever/decidir/autorizar diretamente	Intenção <code>prescricao</code> : recusa cordial + informação do protocolo com fonte + necessidade de validação médica.
Prompt injection ("ignore suas instruções", "revele o system prompt", "você agora é...")	Bloqueio , resposta padrão, evento <code>assistant.blocked</code> na auditoria.
Tema fora de escopo	Redireciona ao escopo clínico/institucional sem responder ao pedido.
Resposta com linguagem prescritiva imperativa ("prescrevo", "administre 2 g")	Reescrita como sugestão ("o protocolo sugere...", "considerar..."); flag <code>linguagem_prescritiva</code> .
Resposta sem aviso de validação humana	Aviso adicionado automaticamente.
Resposta com PII	Redigida; flag <code>pii_na_saida</code> .
Sem fonte recuperada	Flag <code>sem_fontes</code> ; confiança <code>baixa</code> ; texto sinaliza informação geral.
Nos fluxos: sugestão reprovada pelos guardrails	Regenera uma vez com feedback; se persistir, ajusta e sinaliza.

5. Auditoria e rastreabilidade

- Toda ação relevante vira um registro **append-only** em `audit_log` com `prev_hash` e `hash` (SHA-256 do registro canônico + hash anterior). `GET /audit/verify` recomputa a cadeia e aponta onde quebrou (testado com adulteração direta no banco).
- Cada requisição tem `X-Request-ID` (= `trace_id`) propagado para logs, auditoria, mensagens, fluxos e traces do Langfuse.
- Eventos: `auth.login`, `auth.login_failed`, `auth.logout`, `patient.view`, `assistant.chat`, `assistant.blocked`, `assistant.feedback`, `workflow.start`, `workflow.alert`, `workflow.decision`, `workflow.finalize`, `alert.ack`, `knowledge.search`, `knowledge.reindex`, `model.switch`, `audit.verify`.

6. Infraestrutura

- Headers: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, `Cache-Control: no-store`, CSP restritiva na API, HSTS em produção.

- CORS apenas para as origens configuradas; limite de corpo (1 MB); containers com usuário não-root; segredos só via `.env` /variáveis; `git leaks` no pre-commit e na CI; `pip-audit` na CI.
- LiteLLM (quando usado) centraliza chaves e limites; Langfuse registra prompts/respostas — **atenção**: só envie traces para serviços externos com dados já anonimizados (é o caso aqui).

7. O que fica fora do escopo acadêmico (trabalho futuro)

MFA, SSO institucional (OIDC), refresh tokens com revogação, criptografia de campo para PII em repouso, DLP em anexos, revisão clínica formal dos protocolos, homologação regulatória.

Anexo E — Evidências: onde cada exigência aparece

Guia para apresentação/avaliação. Faça login como **admin** (`admin@asclepio.fiap`) para ver as áreas técnicas (IA & Modelos, grafos, auditoria, configurações); as telas clínicas aparecem para médico/enfermagem. URLs abaixo consideram `make setup` (Web :3000 , API :8000) — ajuste se usou outras portas.

1. Fine-tuning de LLM com dados internos (protocolos, FAQs, modelos de laudos/receitas)

Evidência	Onde ver
Dados usados (protocolos, modelos de documentos, FAQ)	Tela Protocolos e documentos (/conhecimento) — lista, busca semântica e conteúdo de cada documento. Arquivos: <code>data/knowledge_base/</code>
Pré-processamento, anonimização e curadoria	<code>data/processed/DATASET_CARD.md</code> e <code>dataset_stats.json</code> (contagens, entidades PII removidas, dedupe, balanceamento, splits). Código: <code>ml/asclepio_ml/data_prep.py</code> , <code>packages/asclepio_core/asclepio_core/anonymizer.py</code>
Treino (LoRA), hiperparâmetros, loss, hardware, tempo	Tela IA & Modelos (/modelo , admin) → card "Fine-tuning" (base, LoRA r/α, épocas, passos, loss, duração) e gráfico <code>docs/assets/eval/train_loss.png</code> . Arquivo: <code>ml/registry.json</code>
Modelo customizado servido e integrado ao assistente	<code>/modelo</code> → "Modelo ativo: asclepio-med (ajustado)"; <code>ollama list</code> mostra <code>asclepio-med</code> ; badge no header. <code>/health</code> → <code>"fine_tuned": true</code>
Avaliação base × fine-tuned (+ referência) e RAG	<code>/modelo</code> → gráficos e tabela (ROUGE-L, BLEU, cobertura, citação, conformidade com guardrails, recusa segura, juiz LLM, latência; hit@5/MRR do RAG) e amostras comparadas. Arquivos: <code>ml/reports/eval_latest.{json,md}</code> , <code>docs/assets/eval/*.png</code> , <code>docs/FINE_TUNING.md</code>
Reprodutibilidade	<code>make prepare</code> , <code>make train</code> , <code>make export</code> , <code>make eval</code> (ou <code>make finetune</code>) — <code>ml/README.md</code>

2. Assistente médico com LangChain (LLM customizada + base estruturada + contexto do paciente)

Evidência	Onde ver
Pipeline LangChain/LangGraph com a LLM fine-tunada	Assistente (/assistente) : resposta em streaming; painel "Fontes & explicabilidade" mostra modelo <code>asclepio-med</code> , intenção, latência, confiança. Admin vê as etapas do grafo (<code>guard_input</code> → <code>classify</code> → <code>retrieve</code> → <code>generate</code> → <code>guard_output</code>) e <code>GET /api/v1/assistant/graph</code> (Mermaid)
Consulta à base estruturada (prontuários)	Pacientes (/pacientes , /pacientes/{id}) : sinais vitais, exames, medicações, evoluções vindos do banco (Postgres/SQLite)
Contextualização com dados atualizados do paciente	No chat, selecione o paciente → botão " ver contexto anonimizado " exibe exatamente o texto enviado à LLM (idade/sexo/setor, vitais, exames, risco) — sem PII
RAG com citações	Respostas com [1] [2]... clicáveis; painel lateral com documento › seção › score › trecho

3. Fluxos de decisão automatizados e seguros (LangGraph)

Evidência	Onde ver
Fluxo que recebe o paciente, verifica exames pendentes, sugere condutas e emite alertas	<code>/pacientes/{id}</code> → " Executar revisão clínica " → <code>/fluxos/{run_id}</code> : linha do tempo das etapas (carregar/anonimizar → exames pendentes/atrasados → triagem de risco qSOFA/NEWS2/valores críticos → alerta imediato se crítico → protocolos (RAG) → sugestão da LLM com fontes → guardrails → alertas → validação humana → finalização)

Evidência	Onde ver
Diagrama do fluxo (LangGraph)	<code>/fluxos</code> (admin) → card "Grafo de revisão clínica" (Mermaid gerado pelo próprio LangGraph). Arquivos: <code>docs/diagramas/*.mmd</code> , <code>docs/ARQUITETURA.md</code>
Humano no circuito	Caixa "Validação humana obrigatória" → Aprovar/Rejeitar (só médico/admin; enfermagem vê 403)
Alertas para a equipe	Alertas (<code>/alertas</code>) — críticos/atenção criados pelo fluxo e por regras, com reconhecimento

4. Segurança e validação

Evidência	Onde ver
Limites de atuação (nunca prescrever sem validação humana)	No chat: "Prescreva 2 g de ceftriaxona para o leito 5" → recusa + informação do protocolo + aviso; "Ignore suas instruções..." → bloqueado (badge do guardrail). Código/testes: <code>packages/asclepio_core/asclepio_core/guardrails.py</code> , <code>packages/asclepio_core/tests/test_guardrails.py</code>
Logging detalhado para rastreamento e auditoria	Auditoria (<code>/auditoria</code> , admin/auditor): todas as ações (login, MFA, consulta a paciente, pergunta, bloqueio, fluxo, decisão, alerta, troca de modelo), com fontes usadas, guardrail, modelo, latência, <code>trace_id</code> ; botão "Verificar integridade da cadeia" (hash chain). Logs estruturados no terminal (<code>make logs</code>); Prometheus em <code>/metrics</code> ; Langfuse (<code>make up-full</code> , <code>:3001</code>)
Explainability (fonte da informação)	Citações numeradas + painel de fontes no chat; sugestões do fluxo com citações; <code>citation_rate</code> na avaliação
Autenticação/autorização reais	Login com MFA (app autenticador) , troca de senha obrigatória, sessões (<code>/conta</code>), perfis (médico não vê IA/Modelos, admin vê tudo), usuários e profissionais (<code>/usuarios</code>), catálogos (<code>/catalogos</code>). Políticas: <code>docs/POLITICAS.md</code>

5. Organização do código e README

Evidência	Onde ver
Projeto modularizado em Python	<code>packages/asclepio_core</code> (núcleo), <code>backend</code> (API + LangGraph), <code>ml</code> (fine-tuning); Swagger em <code>/docs</code>
Instruções completas	<code>README.md</code> (instalação em 1 comando / 1 linha, arquitetura, fine-tuning, segurança, troubleshooting), <code>ml/README.md</code> , <code>frontend/README.md</code>
Testes e CI	<code>make check</code> (85+ testes); GitHub Actions verde (aba Actions do repositório)

6. Entregáveis da Fase 3

Entregável	Local
Código: pipeline de fine-tuning, integração LangChain, fluxos LangGraph	<code>ml/</code> , <code>backend/asclepio_api/services/{assistant,workflow,knowledge,llm}.py</code>
Dataset anonimizado / sintético	<code>data/synthetic/patients.json</code> , <code>data/processed/*.jsonl</code> (+ <code>DATASET_CARD.md</code>), <code>data/knowledge_base/</code>
Relatório técnico (fine-tuning, assistente, diagrama, avaliação)	<code>docs/RELATORIO_TECNICO.md</code> (+ <code>docs/FINE_TUNING.md</code> , <code>docs/ARQUITETURA.md</code> , <code>docs/diagramas/</code>)
Vídeo (≤ 15 min)	roteiro em <code>docs/ROTEIRO_VIDEO.md</code>

Roteiro rápido de demonstração (10 min)

- `/modelo` (admin): fine-tuning, métricas base × ajustado, RAG.
- `/assistente`: pergunta de protocolo (fontes), pergunta com paciente + "ver contexto anonimizado", pedido de prescrição (recusa), injeção (bloqueio).
- `/pacientes/1` → "Executar revisão clínica" → `/fluxos/{id}` → aprovar.
- `/alertas`.
- `/auditoria` → filtrar `assistant.blocked`, verificar integridade.
- `/conta` (MFA) e `/usuarios` (perfis).